Stinger: A Light-Weight Website Fingerprinting Defense Through Poisoning Packet Sequences

Lihai Nie, Xiaodong Dong, *Member, IEEE*, Lili Shi, Laiping Zhao, and Zheli Liu

***Abstract*—Website Fingerprinting (WF) attack can be mitigated through *random camouflage* or *pair camouflage*. *Random camouflage* inserts random dummy packets into the traces according to pre-defined rules. It can be compromised easily by machine learning-based WF attacks. *Pair camouflage* obfuscates the distinguishing features of paired websites by inserting elaborated perturbations into raw traces, thereby misleading the attacker. It is costly in maintaining a perturbation generator for each pair of websites. Based on these insights, we propose *Stinger*, a novel data poisoning based WF defense, which enables effective defense against WF attacks with low bandwidth overhead and only maintains one generator for all websites. *Stinger* exploits the idea of poisoning by contaminating the model directly in such a way that the WF attacks only classify based on the inserted poison sequences, thus being low overhead and website independent. We experimentally evaluate *Stinger* using the DF and AWF datasets. The results show that *Stinger* improves the successful defending rate by an average of 20.37% and 22.83% while reducing overhead by 85.88% and 81.35%, respectively.**

***Index Terms*—Website fingerprinting, data poisoning, Tor.**

I. Introduction

ANONYMOUS communication techniques enhance the privacy and confidentiality of the users' browsing activities on the Internet. ***The onion router* (*Tor*)**, for instance, anonymizes internet traffic using multiple-layer encryption system and currently serves over two million daily users [1]. However, ***Website Fingerprinting (WF)*** attacks [2], [3], [4], [5], [6], [7], [8], [9], [10] impose severe threats to the privacy of *Tor* users. These attacks involve intercepting traces generated during users' browsing via *Tor* and inferring destination websites by analyzing/classifying the collected traces. Recent deep learning-based WF attacks have demonstrated classification accuracy of over 90% (including *DF* [4], *TF* [5], *Var-CNN* [7], *AWF* [9], *GANDaLF* [10]), with the DF attack [4] achieving up to 98% accuracy. This significantly compromises *Tor*'s ability to protect user privacy.

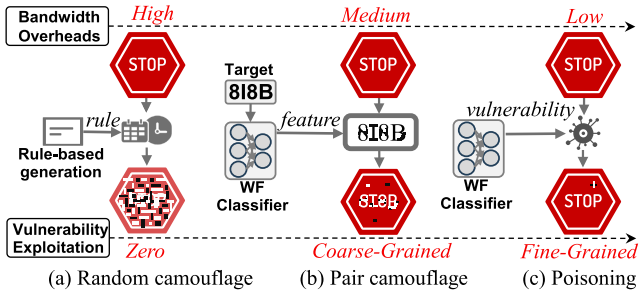


Fig. 1. WF defenses classification in terms of dummy packet generation. To better illustrate the difference between WF defenses, we use the traffic symbol STOP to represent the traces generated when the *Tor* user visits a website.

To defend against WF, defenders insert perturbations (dummy packets) into raw traces to compromise WF classifiers. There are two steps in such WF defenses: (*i*) *Schedule generation* devises a plan for packet insertion, determining the number of dummy packets and their insertion positions and (*ii*) *Implementation* entails the actual injection of dummy packets into the raw traces according to the schedule. We summarize the existing WF defense techniques [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22] and find that they commonly share the basic idea of *camouflage*, i.e., disguising themselves as something else through the combination of raw traces and the pre-set dummy packets.

As illustrated in Fig. 1, emcamouflage can be further divided into *random camouflage* and *pair camouflage*. In the *random camouflage* paradigm [12], [17], [20], [21], [22], *akin to soldiers wearing camouflage uniforms*, the trace features for visiting a website are obfuscated by the rule-based random perturbation (Fig. 1(a)). As a result, the camouflaged traces of different websites may exhibit similarities or significant variations. Random camouflage is challenged by the trade-off between the camouflage budget (i.e., the number of inserted dummy packets) and the camouflage effect: a larger amount of inserted packets will enhance concealment but inevitably lead to substantial bandwidth overhead, thereby degrading both the user experience and website performance, and vice versa.

Received 10 February 2025; revised 9 August 2025 and 26 November 2025; accepted 11 December 2025. Date of publication 24 December 2025; date of current version 5 January 2026. This work was supported in part by the Youth Project of the National Natural Science Foundation of China under Grant 62402248; in part by the Postdoctoral Fellowship Program (Grade C) of China Postdoctoral Science Foundation under Grant GZC20240756; in part by China Postdoctoral Science Foundation-Tianjin Joint Support Program under Grant 2023T015TJ; in part by the Open Research Project of the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD025; and in part by the Natural Science Foundation of Shandong Province under Grant ZR2022LZH014. The associate editor coordinating the review of this article and approving it for publication was Prof. Albert Levi. *(Corresponding authors: Laiping Zhao; Xiaodong Dong.)*

Lihai Nie is with the Key Laboratory of Data and Intelligent System Security (DISSec), Ministry of Education, College of Cryptology and Cyber Science, and the Academy for Advanced Interdisciplinary Studies (AAIS), Nankai University, Tianjin 300350, China, and also with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences), Jinan, Shandong 250353, China (e-mail: nlh@nankai.edu.cn).

Xiaodong Dong and Zheli Liu are with the Key Laboratory of Data and Intelligent System Security (DISSec), Ministry of Education, College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China (e-mail: dongxiaodong@nankai.edu.cn; liuzheli@nankai.edu.cn).

Lili Shi and Laiping Zhao are with Tianjin Key Laboratory of Advanced Networking (TANKLab), College of Intelligence and Computing (CIC), Tianjin University, Tianjin 300354, China (e-mail: llshi66@tju.edu.cn; laiping@tju.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3646493

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

In the *pair camouflage* paradigm [11], [18], [19], [23], *like chameleons change the skin color in response to their environments*, defenders perturb the traces of sensitive websites by mimicking that of the target website, thereby evading identification by the WF classifiers (Fig. 1(b)). *Pair camouflage* can reduce the bandwidth overhead since it solely mixes pair-wise flows. However, the schedule generator is website-dependent, indicating it has to maintain a unique perturbation generator for each pair of websites. This definitely imposes heavy burdens for the long-term WF defense scenarios.

Both *random camouflage* and *pair camouflage* belong to the *primary defenses*, serving as the initial barriers to prevent websites' traces from being recognized by WF attackers. To achieve advanced WF defense with the advantages of low bandwidth overhead and low model cost, we explore a *secondary defense* method called *poisoning defense* (Fig. 1(c)), which defends against the WF attacks through poisoning the WF classifier. It does not significantly alter the raw trace, but just disables the WF classifier by triggering the corresponding vulnerabilities using a short poisoning packet sequence.

However, there are two major challenges when designing such a poisoning-based advanced WF defense.

(1) *How to identify the vulnerability of an unknown WF classifier?* Although the vulnerability of a known attack classifier can be identified through adversarial learning [14], it relies on a strong assumption that the model structure and parameters are given in advance. As it is nearly impossible to get this information from WF attackers, we cannot adopt the adversarial learning. To solve this problem, we exploit the common *overfitting* phenomenon existing in deep learning models. That is, beguiling the WF classifier to "over-fit" on the elaborated poisoning packet sequences. Then the classifiers are compromised once the poisoning sequences change.

(2) *How to build poisoning method based on the vulnerability?* Although *overfitting* is commonly existing in deep learning, it is still difficult to automatically generate "poison sequence", which can guide a black-box WF classifier to *over-fit*. We solve this problem by using the idea of model transferability, that is, the different models share the same or similar vulnerabilities. In particular, we co-train a generative model and a surrogate WF classifier by configuring the generative model to produce the poisoning sequences aiming to lead the WF classifier over-fits. Then the poisoning sequences can also compromise the real WF attacks due to model transferability.

To overcome these challenges, we present *Stinger*,[1] a data poisoning WF defense, which is able to effectively defend against existing WF attacks with minimal overhead. *Stinger* provides a new defense idea for WF defending field: defending against WF attacks by actively attacking them.

Different from prior defenses that try to obfuscate the features of website traces, *Stinger* does not change the website traces significantly except for the implantation of the poisoning sequences. Hence, it can defend against WF attack with little bandwidth overhead.

The contributions of this paper are summarized as follows:

- We observe the defense performance dilemma of existing WF defenses and find that actively attacking WF classifiers can reduce overhead and improve defending effect.
- Based on the observation, we propose *Stinger*, a proactive WF defense involving two key phases: (*i*) inducing WF classifier over-fits on specific poisoned packet sequences and (*ii*) deceiving the over-learned WF classifier by shuffling poisoning sequences using the user-side secrets.
- We evaluate *Stinger* on real-world traces and compare its performance with state-of-the-art methods concerning defensive effectiveness and overheads. The experimental results show that *Stinger* achieves significantly better defensive efficiency than competing defenses, with a higher successful defending rate and reduced bandwidth overhead. Specifically, *Stinger* improves the successful defending rate by an average of 32.44% and 22.33% while reducing bandwidth overhead consumption by 75.71% and 83.85% on DF and AWF dataset, respectively.

The rest of the paper is structured as follows. We study the existing WF attacks and defenses (§ II), and use the findings to design *Stinger* (§ IV). Then we evaluate *Stinger* (§ VI), discuss (§ VII), summarize related work (§ VIII), and conclude (§ IX).

TABLE I
ACCURACY EVALUATION OF THE PRIOR WF ATTACKS

Category	WF Attack	Undefended∗	Defended‡
Traditional Machine Learning	*K-NN* [3]	95.00%	8%
	k-FP [6]	95.50%	21%
	CUMUL [2]	97.30%	18.74%
Deep Learning	*DF* [4]	98.30%	29.36%
	Var-CNN [7]	97.21%	24.03%
	GANDaLF [10]	94.07%	20.41%

Note: ∗ The accuracy is obtained on the raw *Tor* traces (the undefended mode); ‡ The accuracy represents the lowest value computed among the available defenses with $<100\%$ bandwidth overheads (the defense mode).

II. PRELIMINARIES

A. WF Attacks and Defenses

In this section, we evaluate the existing WF attacks and defense schemes on real-world *Tor* traces. The results demonstrate that prior camouflage-based defense schemes often encounter the *defense dilemma*, as they cannot achieve high defending effects with relatively low bandwidth overheads.

1) WF Evaluation: We evaluate existing WF attacks in terms of attacking accuracy, defined as the ratio of correct predictions to all predictions. Furthermore, to measure attacking accuracy in different scenarios, we conduct WF attacks on raw *Tor* traces and defended traces, respectively. We only present the worst performance on the *Tor* traces defended by different defense methods. For the implementation of WFs, we set all hyperparameters following the original recommendations [2], [3], [6], [7], [10]. The results are presented in TABLE I. Moreover, we make the following observations from TABLE I.

Observation #1: *The available defense schemes can effectively defend against the existing WF attacks, and they decrease the attacking accuracy by an average of 75.97%.*

As we can see, all WFs achieve considerably high performance on undefended traces, e.g., the accuracy of *CUMUL* and *DF* achieves up to 97.3% and 98.3%, indicating they can nearly identify all undefended traces. On the other hand, the *WFs*' performance dramatically drops on the defended traces. For example, the accuracy of *K-NN* drops from 95% to 8% once the traces are defended. This demonstrates the available defenses can effectively defend against the existing *WFs*.

However, there are still two challenges for the existing defenses. First, their performance is discounted by deep

[1]The source code is available at *https://github.com/sxy888/Stinger*

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

TABLE II
A OVERALL COMPARISON BETWEEN STINGER AND PREVIOUS DEFENSES

Category	WF Defense	Overhead	*SDR**	Year
Random Camouflage	*Tamaraw* [20]	234.33%	92.13%	2014
	WTF-PAD [21]	61.46%	10.03%	2016
	FRONT [17]	62.89%	22.66%	2020
Pair Camouflage	*Mockingbird* [19]	59.47%	62.84%	2021
	ALERT [24]	20.34%	60.72%	2024
Poisoning	*Stinger* [this paper]	12.38%	72.14%	2025

Note: *SDR** is measured in the worst case among all *WF* defenses. This is because attackers can arbitrarily choose *WF* schemes to launch attacks.

learning-based WFs, which can achieve nearly 30% accuracy on the defended traces. Moreover, the existing defenses achieve high performance by consuming high bandwidth overheads, inevitably degrading the user experience. To this end, we focus on analyzing the trade-off between defense effectiveness and bandwidth overheads when evaluating the existing defenses.

2) Defense Evaluation: Prior WF defenses share a similar idea of *camouflage*, i.e., inserting dummy packets into raw traces to make them resemble other traces and thereby evade detection by classifiers. Moreover, they camouflage in different ways. *Random camouflage* generates insertion scheme according to pre-defined rules. They are static and easy to implement but can be easily compromised by deep learning WFs. Examples of *random camouflage* include *Tamaraw* [20], *WTF-PAD* [21] and *Front* [17]. *Pair camouflage* aims to reduce the bandwidth overhead generated in camouflage. It mixes pair-wise traces using gradients and predictions from WF classifier. Examples of *pair camouflage* include *ALERT* [24] and *Mockingbird* [19]. To measure the existing defenses, we define a metric *Successful Defending Rate* (*SDR*), formulated as 1-*accuracy* since *SDR* and WFs' accuracy are mutually exclusive. The SDR of defenses is presented in TABLE II.

Observation #2: *The existing WF defenses are inefficient in saving overheads, which range from 20.34% to 234.33%.*

In particular, random camouflage- and pair camouflage-based defenses on average consume 119.62% and 61.78% overheads, respectively. Those are significantly higher than the method proposed by this paper. This is because all camouflage-based methods belong to the primary defense, which focuses on masking features of raw traces (randomizing or mixing pair-wise traces) without exploiting fine-grained vulnerabilities. As a result, they fail to improve *SDR* with fewer overheads.

Moreover, we also find that the higher *SDR* can be achieved by consuming more overheads for defenses within the same category. Taking *random camouflage* as an example, *Tamaraw* presents much higher *SDR* than *FRONT*, *WTF-PAD* using extra 172.16% overheads. The reason for such trade-off is straightforward: more dummy packet insertions will mask more distinctive features of raw traces, thereby leading to the higher *SDR*. We call the trade-off between *SDR* and bandwidth overheads *defense dilemma*: high *SDR* can be achieved only by consuming prohibitively high bandwidth overheads.

Therefore, *SDR* alone cannot serve as a reliable evaluation metric for *WF* defenses, since it can be improved by inserting more dummy packets. To comprehensively evaluate the existing defenses, we measure the defense dilemma of the existing methodology in the next section.

B. The Defense Dilemma

In this section, we evaluate the existing defense schemes' dilemma in terms of *SDR* and bandwidth overheads.

1) Scheme Selection: We implement the two representative WF attacks: *CUMUL* [2] and *DF* [4], which are the state-of-the-art ML- and DL-based WF attacks, respectively. For defense schemes, we choose to implement *FRONT* [17], *Tamaraw* [20] and *Mockingbird* [19]. The reasons are presented as follows. *FRONT* and *Tamaraw* are the state-of-the-art methods of random camouflage in opposite manners. Specifically, *FRONT* manipulates the traces of the same website to be different, while *Tamaraw* makes the traces of different website to be similar. Besides, *Mockingbird* is selected as it is the representative pair camouflage method with the considerably high performance.

2) Overhead Controlling: FRONT uses parameter N_c and N_s to control bandwidth overheads (i.e., the number of dummy packets). In particular, the overhead at client/server end is a random value uniformly sampled from range $(1, N_c/N_s)$. Thus, the averaged overhead of *FRONT* is $(N_c + N_s)/2 + 1$.

Tamaraw uses P_{in}/P_{out} to control the time interval of outgoing/incoming packets, further impacting the bandwidth overheads. From the qualitative perspective, a large time interval between packets will lead to higher bandwidth overheads.

Mockingbird uses parameter α to control the overhead. Specifically, α is the learning rate of the gradient-based optimization, which minimizes the distance between the trace and the target. A larger α indicates it takes bigger steps toward the target, resulting in more dummy packets being inserted [19].

3) Dilemma of Random Camouflage: To measure the relationship between bandwidth overheads and defense effect, we use *FRONT* to defend against WFs (*CUMUL* and *DF*) by varying N_c in {480, 600, 720, 840} and varying N_s in {1120, 1400, 1680, 1960}. As shown in Fig. 2(a) and Fig. 2(b), there is an obvious trade-off between bandwidth overheads and the attacking accuracy of WFs. In particular, *FRONT* reduces the accuracy of *CUMUL* to $< 32.9\%$ when it inserts $> 35.96\%$ dummy packets. Hence, *FRONT* is relatively effective in defending against the *CUMUL* attack. In case of *DF*, *FRONT* also reduces the accuracy of *DF*, but the accuracy only declines to 77.3% when the overheads achieves $> 62.9\%$.

Moreover, we use *Tamaraw* to defend against WF attacks by varying $p_{out} \in \{0.04, 0.03, 0.02, 0.01\}$ and $p_{in} \in \{0.008, 0.007, 0.006, 0.005\}$. As shown in Fig. 2(c) and Fig. 2(d), the trade-off is also obviously presented. That is, *Tamaraw* can defend against *CUMUL* and *DF* by decreasing accuracy $< 3.91\%$ and $< 3.77\%$ with the bandwidth overhead of 339.04%. However, the attack accuracy of DF increases to 19.2% when the bandwidth overhead decreases to 166.61%.

4) Takeaway Message: Random camouflage-based defense is highly practical because (i) its rule-based schedule generation is simple and easy to realize and (ii) the frequent updates and modifications of website flows do not necessarily require schedule updates. However, it is either easily defeated by WF attacks or incurs significantly high bandwidth overhead.

5) Dilemma of Pair Camouflage: We further conduct four groups of experiments to evaluate the defense ability and bandwidth overhead of the *pair camouflage-based Mockingbird* [19]. We control the bandwidth overhead of *Mockingbird* through choosing $\alpha \in \{1, 3, 5, 7\}$. As shown in Fig. 2(e) and Fig. 2(f), compared with the *random camouflage* methods, the *pair camouflage* based *Mockingbird* [19] can further reduce the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

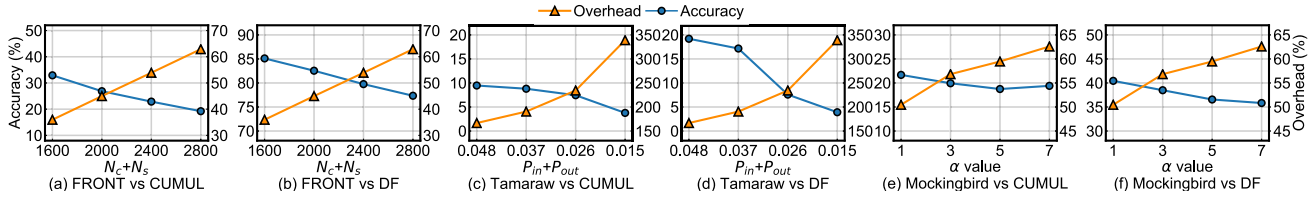


Fig. 2. Trade-off between bandwidth overheads and attacking accuracy when using camouflage-based defense.

identification accuracy of WF attacks. For example, *Mockingbird* can reduce the accuracy of *CUMUL* and *DF* to < 21.68% and 40.44%, respectively. Under the same bandwidth overhead = 50%, the accuracy under *Mockingbird* is about 40%, while the accuracy under *FRONT* is > 80%.

In addition, the *defense dilemma* also exists in *pair camouflage*. Fig. 2(e)and Fig. 2(f) show that, as the α increases, the identification accuracy of both *CUMUL* and *DF* decreases while the bandwidth overhead increases. The perturbations generated by pair camouflage can be either trace-aware or trace-agnostic. Generating trace-aware perturbations [19] requires feedback information from the classifier and websites related information to compute schedule strategies. However, Juarez et al. [25] point out that most websites are updated and modified periodically, and we count the update frequency of news websites and find that they update every 10 to 20 minutes. It means that such defenses need to collect new labeled dataset to update their defense schedules regularly to keep high defense effect, which greatly increases the data collection efforts. What is more, generating trace-agnostic perturbations [18] is also costly since defenders need to maintain an individual generator for each pair of websites.

6) Takeaway Message: Pair camouflage-based WF defenses are more effective than the random camouflage ones because (i) they are able to defend against existing WF attacks through adversarial learning; (ii) the bandwidth overhead becomes lower. However, pair camouflage still falls into the defense dilemma. Moreover, pair camouflage is less practical due to the computational cost of maintaining perturbation generators and the overhead associated with adapting to website updates.

C. Secondary Defense

The *defense dilemma* demonstrates that it is a challenge to improve the camouflage effect without increasing bandwidth overheads. This challenge arises because all camouflage-based methods belong to *primary defense* [26], which primarily aims to mask the identification information of the raw *Tor* traces. Clearly, introducing more perturbations will improve imperceptibility, but leads to higher bandwidth overhead costs.

Despite that falling *defense dilemma*, we observe *pair camouflage* can partially mitigate the defense dilemma compared to *random camouflage*. For example, *FRONT* (random type) makes *DF*'s accuracy drop to 77% with 70% bandwidth overhead (Fig. 2(b)). In contrast, *Mockingbird* (pair type) can reduce *DF*'s accuracy < 36% with 63% bandwidth overhead.

The performance gap between the two defenses can be analyzed in terms of vulnerability exploitation. In particular, random camouflage only randomizes the traces without utilizing any classifier vulnerability (*zero-vulnerability*). Since the randomization process is independent of classifier, it will waste the majority of bandwidth overheads when defending WF attacks. Pair camouflage utilizes the classifier feedback (outputs and gradients, *coarse grained-vulnerability*) to optimize its generative network to produce more similar traces to the target one. Compared to random method, pair camouflage is more defense-oriented, thereby saving the bandwidth overheads. This inspires us to exploit more fine-grained vulnerabilities of WF classifiers (e.g., weaknesses/backdoors) to achieve more advanced WF defense, which is expected to further mitigate the defense dilemma of camouflage-based methods.

We explore the possibility of *secondary defenses* (or *fighting defense* [26]) to mitigate the *defense dilemma*, i.e., not camouflaging the raw traces but instead fighting against the WF classifier by exploiting fine-grained vulnerabilities. One typical and effective fighting defense method is *data poisoning* [27], [28], [29], which involves inserting crafted poison samples into the training set, thereby contaminating the model trained on such a poisoned set. As the poison sample does not perturb the entire raw trace but solely insert a small piece of poisoned packet sequence, it is expected to achieve high defense effectiveness with costing low overheads.

In practice, the WF defender has no knowledge of the WF classifier (i.e., a black-box model). Consequently, it is still challenging to discover the vulnerabilities of such a black-box classifier. Fortunately, there is an opportunity to partially control attackers' classifiers in WF scenarios. Specifically, WF attackers commonly visit the target websites via *Tor* and collect the related traces as training sets. Therefore, we can insert the poisoned samples into training sets for WF attackers. Furthermore, it becomes possible to mislead the WF classifier through poisoning the raw traces, and the contaminated WF classifier can be easily compromised by the poisoned samples.

III. Threat Model

A. Adversary Assumption

Firstly, we assume the adversary to be a local and passive eavesdropper. Here, *local* indicates that the adversary sits between clients and entry nodes of the *Tor* network, while *passive* means that the adversary can only eavesdrop on *Tor* packets but is prohibited from manipulating, dropping, or delaying any of them. Moreover, the adversary is allowed to parse the flows of various websites and isolate them from the other types of *Tor* traffic (e.g., *APPs*' flow). Besides, the adversary extracts directional sequence from the raw traces and trains *WF* classifier on the eavesdropped traces. Finally, we assume the adversaries silently collect traffic traces, and WF defenders cannot detect them or monitor their activities.

B. Defense Description

Following the prior defenses [18], [19], [20], we insert dummy packets into the raw traces to confuse WF adversaries. To achieve this, we need to deploy two proxies on *Tor* clients and delays to cooperatively manipulate dummy packets defined by defenses. In particular, the proxy on clients inserts outgoing packets and drops the incoming ones. The proxy on

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

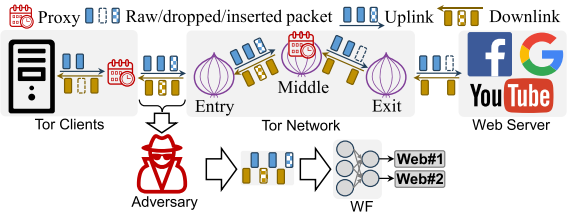


Fig. 3. The threat model.

Tor relays plays the opposite role. For a typical *Tor* network, there are three delay nodes: the entry, the middle and the exit. We select to deploy the proxy on the middle node because (*i*) the entry is possibly controlled by adversaries [17] and (*ii*) proxy on the exit will increase the impacts of defenses (dummy packets can be dropped at middle nodes in advance). Moreover, we assume that the adversary can compromise a subset of middle nodes in the *Tor* network, enabling them to collect a mixed set of training traces containing both raw and *Stinger*-defended traffic. We depict the threat model in Fig. 3.

IV. Stinger Design

In this section, we sequentially present *Stinger's* overview, detailed design and deployment in the real-world scenario.

A. Overview

As depicted in Fig. 4, itStinger's key idea is to guide WF classifiers to over-fit on a particularly crafted packet sequence by manipulating training samples (*misleading*) and to disable the overfitted *WF* classifier by changing that sequence (*deceiving*).

1) Misleading Stage: In this stage, *Stinger* aims to produce such a poison packet sequence ($\mathcal{P}$ sequence), which can guide *WF* classifiers to over-fit on it (Fig. 4(a)). To achieve this, we jointly train a poison model ($\mathcal{P}$ model) and a deep WF classifier as follows. ❶: $\mathcal{P}$ model accepts $\mathcal{P}$ labels and outputs $\mathcal{P}$ sequence candidates. ❷: Randomly select traces from the repository that records the traces of the visiting different websites and then defend the selected traces by inserting $\mathcal{P}$ sequence candidates. ❸: The defended traces are fed into a WF classifier, which subsequently outputs the predictions of $\mathcal{P}$ labels. ❹: Jointly optimize $\mathcal{P}$ model and deep *WF* classifier by minimizing the loss between the $\mathcal{P}$ label and its prediction. Once the loss converges, we can claim that the *WF* classifier accurately predicts the $\mathcal{P}$ label regardless of the raw trace. Since $\mathcal{P}$ label is the identity of $\mathcal{P}$ sequence, we can also claim the converged *WF* classifier over-fits on the $\mathcal{P}$ sequences.

2) Deceiving Stage: In this stage, *Stinger* aims to deceive an over-learned classifier by shuffling $\mathcal{P}$ labels using a secret mechanism. A *secret* defines a fixed map between $\mathcal{P}$ labels and the website identities, and it differs among different users (including adversaries). The deceiving process is described as follows. First, the adversary collects network traces defended by *Stinger* and trains its WF classifier. At this time, adversary cannot verify whether its WF classifier over-fits on the $\mathcal{P}$ sequences since each website corresponds to a fixed $\mathcal{P}$ sequence during the model training and validating. Thus it can be observed that the validating accuracy approaches the training accuracy, although the adversary's classifier over-fits. However, the overfitted classifier can be easily compromised when the adversary launches attack. Specifically, the secrets differ between victims and adversaries, indicating that a $\mathcal{P}$ label corresponds to different websites for the victim and the adversary. Moreover, the overfitted classifier predicts websites only depending on the $\mathcal{P}$ sequences, causing the adversary to fail to predict websites that the victim visits.

B. Misleading Design

In this section, we describe the detailed misleading designs in $\mathcal{P}$ sequence generation and optimization, respectively.

1) Sequence Generation: We utilize a neural network-based poison model to yield $\mathcal{P}$ sequence, which is formulated as

$$Seq_p = G_p^{\phi}(Lab_p), \tag{1}$$

here Lab_p, G_p^{ϕ} and Seq_p is the poison label, the poison model parameterized by ϕ and the poison sequence, respectively. Eq. 1 represents the process of transforming the poison label Lab_p into the poison sequence Seq_p via a generator G_p^{ϕ}.

Poison label plays two roles during poison sequence generation. On the one hand, it is the seed of the poison model, determining the poison sequence and outputs of WF classifier. Thereby, *Tor* users can be protected from WF attacks by exploiting poison labels different from adversaries. On the other hand, poison label is the ground-truth of WF classifier's output. In particular, the fact that WF classifier over-fits on poison sequences indicates its outputs are poison sequence-specific or it can correctly classify poison sequences regardless of *Tor* traces. Moreover, poison label is the identity of poison sequence. Thus the easy but effective way of training poison model is to consider poison label as the ground-truth of classifier's output and minimize their difference. This requires poison label to keep the same format as classifier's output.

Poison model maps a poison label to a poison sequence. We choose neural architecture to implement it since it can be cooperatively optimized with WF classifier. There are several basic neural networks, including MLP, LSTM, CNN and even transformer-based models, like GPT and LLaMA. We implement the poison model using MLP for the following reasons. Poison sequence generation does not involve local feature extraction and recurrent generation. Thus LSTM and CNN are not suitable for tasks. Moreover, although transformer-based large language models can settle this problem, it is too costly for *Tor* users and defenders to run such large models. The specific structure of the poison model is depicted in Fig. 5.

Poison sequence triggers the WF classifier to output a particular output regardless of the rest of the *Tor* traces. Here we parameterize poison label with its length l, which is utilized to control bandwidth overheads and further to measure the impact on defense effectiveness. To make poison sequence work, we need to insert it into the raw trace, which is formulated as

$$Tra^* = Tra \oplus Seq_p, \tag{2}$$

where Tra and Tra^* represents the raw *Tor* trace and the defended trace, and $\oplus$ refers to insertion operation. Eq. 2 represents the process of combining raw traces with $\mathcal{P}$ sequences to generate the defended traces. In particular, we propose two insertion operations as follows. At misleading stage, the delay and overheads are not involved (we only focus on model learning), thus we insert the entire poison sequence. At deceiving stage, we utilize *delay*- and *overhead*-saving scheme, where the dummy packet is not inserted if the current packet matches the format of the poison sequence.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

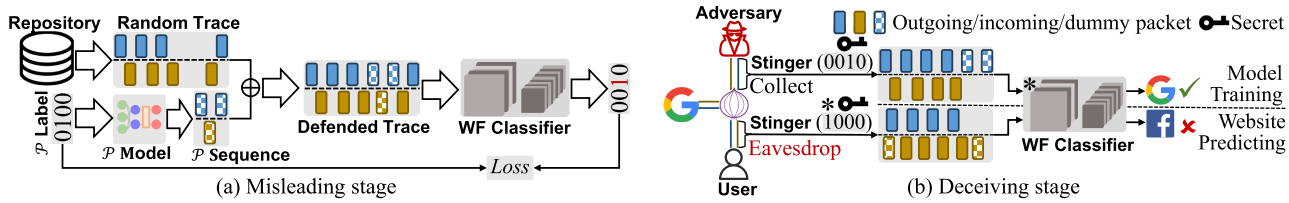


Fig. 4. *Stinger* overview. Outgoing packets are transmitted from *Tor* clients to website servers; Incoming packets are transmitted in the opposite direction.

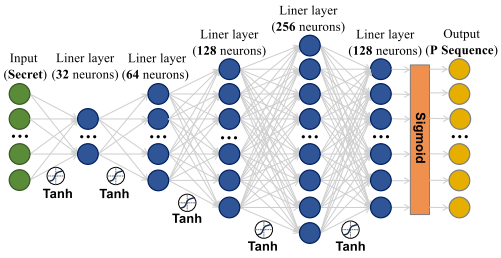


Fig. 5. $\mathcal{P}$ model's structure.

2) Sequence Optimization: Since the defender is unaware of the WF classifier, it is challenging to evaluate a poison sequence candidate and further optimize the poison model. Fortunately, there are two chances to achieve this in WF defending scenario. Firstly, the defender is able to manipulate the training sets of the adversary, who is assumed to train their WF classifier on the collected *Tor* traces. Moreover, the vulnerabilities of deep architecture are commonly shared by various models. The above chances enable the defender to exploit surrogate deep classifier to evaluate the traces containing poison sequences, which is formulated as

$$\hat{\mathcal{P}} = WF_{\psi}(Tra^{*}), \tag{3}$$

where $WF_{\psi}(\cdot)$ represents the surrogate classifier parameterized by ψ and $\hat{\mathcal{P}}$ refers to the output of WF_{ψ}. Eq. 3 represents the process that the classifier WF_{ψ} outputs the prediction $\hat{\mathcal{P}}$ based on the perturbed trace Tra^{*}. Moreover, we select raw DF model as the surrogate classifier for the following two reasons. Firstly, DF model is one of the state-of-the-art WF methods. The vulnerabilities mined from it are expected to generalize well to other methods. Moreover, compared with learned models, an unlearned model is easier to mine its vulnerabilities. The poison model can be optimized by}

$$\underset{\phi,\psi}{\operatorname{argmin}}\, \mathcal{L}(\mathcal{P}, \hat{\mathcal{P}}), \tag{4}$$

where loss $\mathcal{L}$ is selected as cross entropy since $\mathcal{P}$ and $\hat{\mathcal{P}}$ are categorical labels. Eq. 4 measures the categorical loss between the poison label ($\mathcal{P}$) and its prediction ($\hat{\mathcal{P}}$). In particular, we choose Adam to minimize Eq. 4, whose optimal parameters and batch size are determined using the grid search.

C. Deceiving Design

As depicted in Fig. 4(b), itStinger deceives a WF classifier by *i*) inducing the classifier to over-fit on $\mathcal{P}$ sequences and then *ii*) shuffling the classifier's outputs by switching $\mathcal{P}$ sequences.

1) Over-Fit Inducing: To induce WF classifiers to over-fit, we poison WF attackers' training sets by inserting the $\mathcal{P}$ sequences into the raw traces. As a result, a fresh WF classifier will over-fit on the $\mathcal{P}$ sequences once trained on such poisoned sets. Poisoning the training sets relies on the following two bases: 1) the permission to manipulate the attacker's training set, and 2) the effective $\mathcal{P}$ sequences that can compromise an unknown fresh model. These bases are analyzed as follows.

Basis #1: As stated in *Threat Model* (§ III), WF attackers collect training sets by visiting target websites on *Tor* and acquiring the generated traces as training data. This process gives defenders chances to manipulate the WF attackers' training sets by deploying agents on *Tor* clients/middle nodes and inserting outgoing/incoming dummy packets, thereby altering the trace patterns observed by the WF attackers. In this way, defenders are capable of inserting $\mathcal{P}$ sequences into WF attackers' training sets, facilitating the over-fit inducing.

Basis #2: Since $\mathcal{P}$ sequences are trained to compromise a surrogate classifier, there is no guarantee that they can guide an unknown WF classifier to over-fit. Fortunately, the prior works [18], [19], [30], [31] show that different deep models share the same backdoor vulnerabilities. That is, adversarial attack generated against one model frequently exhibit transferability to architecturally different models trained on the same task [32], [33]. This transferability phenomenon stems from two key factors: (1) deep models often incorporate similar fundamental building blocks (e.g., convolutional or recurrent units), and (2) these shared components tend to possess consistent vulnerability patterns across implementations. Thus, we train $\mathcal{P}$ sequences against such surrogate classifier containing various units. Then $\mathcal{P}$ sequences can lead any deep classifier containing such units to over-fit. In Evaluation, we verify the vulnerability transferability across different models by varying *Stinger's* surrogate classifiers and the real WFs.

Based on the above foundations, WF defenders can cause an unknown fresh WF classifier to over-fit on specific patterns by inserting $\mathcal{P}$ sequences into the attackers' training datasets.

2) Output Shuffling: Since the overfitted classifier predicts only relying on the $\mathcal{P}$ sequences, the defender can control the classifier's outputs by inserting different $\mathcal{P}$ labels. This enables the defense against WF attacks by assigning different $\mathcal{P}$ labels for different users, e.g., potential victims and adversaries. Specifically, we design a secret mechanism to assign $\mathcal{P}$ labels.

Secret formulation: A secret defines a map between $\mathcal{P}$ labels/sequences and sensitive websites, which is formulated

$$\begin{aligned} &s : \mathcal{P}' \rightarrow W, \\ &s.t. = W = \{w_1, \cdots, w_i, \cdots, w_m\}, \\ &\mathcal{P} = \{p_1, \cdots, p_i, \cdots, p_n\}, \\ &\mathcal{P}' \subset \mathcal{P}, \& \ |\mathcal{P}'| = |W|, \\ &m \ \leq n, \end{aligned} \tag{5}$$

where s signifies a secret and $W/\mathcal{P}$ represents the website/poison sequence collection, m and n is the number of protected websites and poison sequences, respectively. Besides, collection $\mathcal{P}'$ is a subset of $\mathcal{P}$ and has the same size as W. Eq. 5 defines the map between $\mathcal{P}$ labels ($\mathcal{P}'$) and websites (W).

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

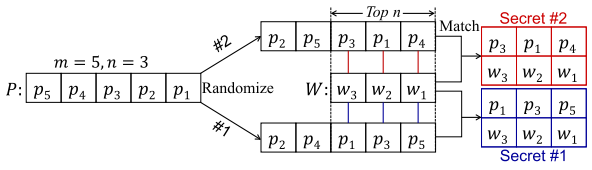


Fig. 6. Illustration of generating secrets.

Secret design: A desirable secret should be user-specific, that is, it keeps fixed for a unique user and differ over various users. The fixed characteristic can ensure the success of over-fit inducing, that is, the WF attacker cannot be aware of over-fitness since the fixed secret can maintain high validating accuracy. While the differing characteristic can help to shuffle output since attackers and victims hold different secrets and thereby the classifier outputs differently for the same website.

To this end, we design random-based secret generation process by randomizing the poison sequence collection $\mathcal{P}$,

$$\tilde{\mathcal{P}} = randomsize(\mathcal{P}),$$
$$\tilde{\mathcal{P}} = \{\tilde{p}_1, \cdots, \tilde{p}_i, \cdots, \tilde{p}_n\}, \tag{6}$$

where $\tilde{\mathcal{P}}$ is randomized $\mathcal{P}$ and $\tilde{p}_{i(1\geq i\geq n)}$ is an element of $\tilde{\mathcal{P}}$. Eq. 6 represents the process of randomly shuffling the order of elements in poison sequence collection ($\mathcal{P}$). The randomizing function is select as `random.randint()` from `NumPy`. Then we extract the top m elements from $\tilde{\mathcal{P}}$

$$\mathcal{P}' = \{\tilde{p}_1, \cdots, \tilde{p}_i, \cdots, \tilde{p}_m\}, \tag{7}$$

where $\mathcal{P}'$ is the shuffled poisoned sequence collection. A secret s is obtained by sequentially matching $\mathcal{P}'$ and W,}

$$s(\tilde{p}_i) = w_i, \quad s.t. \quad 1 \leq i \leq n. \tag{8}$$

Eq. 8 matches the top n elements in $\mathcal{P}'$ with the sensitive websites sequentially, which is illustrated in Fig. 6.

The secret generation mechanism can flexibly accommodate the incremental websites. In particular, when the number of protected websites increases by i, the maps are correspondingly extended by matching the top $n+i$ $\mathcal{P}$ sequences with $n+i$ websites. However, this approach remains viable only when n does not exceed m, that is, the number of protected websites must not surpass the available $\mathcal{P}$ sequences. Once this condition is violated, *Stinger* would require either retraining or incremental training to expand its pool of $\mathcal{P}$ sequences.

Using the above random-based generation method, there are inevitably conflicting maps between victims and WF adversaries. Specifically, *conflicted maps* refer to identical maps that occur in two secrets owned by different users. These conflicting maps diminish *Stinger's* performance since WF classifiers can fingerprint parts of protected websites. To measure the impact of conflicting maps, we analyze their theoretical expectations and find that it can be controlled to be less than 1 by setting $n/m \leq \sqrt{e}$. The detailed analysis is presented as follows.

Theorem 1: The expectation of conflicted maps' number between two random secrets is smaller than $e^{n/m} - 1$.

Proof: Denote by X the number of conflicted maps between two random secrets. Its expectation is computed as

$$\mathbb{E}(X) = \sum_{x=1}^{x=n} P_r(X = x) * x, \tag{9}$$

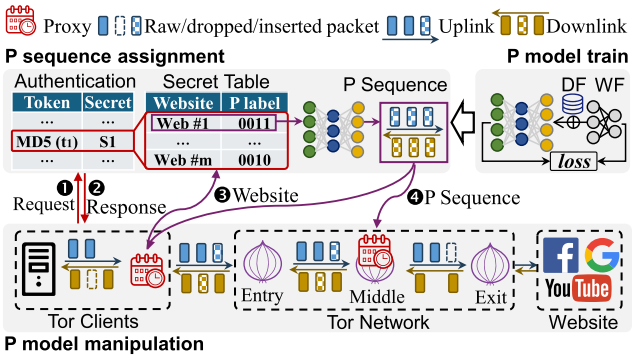


Fig. 7. Illustration of deploying *Stinger*. The *Tor* client synchronizes the initialized/updated token with the middle node, which is omitted for simplicity.

where $P_r(X = x)$ represents the probability of x conflicted maps are observed in two secrets, which can be derived as

$$P_r(X = x) = \begin{cases} P_r(X \geq x) - P_r(X \geq x+1), & if\ x < n, \\ 1, & if\ x = n, \end{cases} \tag{10}$$

where $P_r(X \geq x)$ represents the probability that the number of conflicted maps exceeds x. It can be derived as

$$P_r(X \geq x) = \begin{cases} C_n^x * A_{m-x}^{n-x}/A_m^n, & if\ x < n, \\ 1, & if\ x = n, \end{cases} \tag{11}$$

where A_m^n represents the number of possible permutations when selecting and arranging n distinct elements from a set of m distinct elements, $C_n^x * A_{m-x}^{n-x}$ refers to number of possible permutations when at least x elements are conflicted. According to Eq. 9, Eq. 10 and Eq. 11, $\mathbb{E}(X)$ is rewritten as

$$\begin{aligned} \mathbb{E}(X) &= \sum_{x=1}^{n-1} (C_n^x A_{m-x}^{n-x} - C_n^{x+1} A_{m-x-1}^{n-x-1})/A_m^n * x + n \\ &= \sum_{x=1}^{n} (C_n^x A_{m-x}^{n-x})/A_m^n \\ &= \sum_{x=1}^{n} A_n^x/(A_m^x x!) \\ &= \sum_{x=1}^{n} \frac{n}{m} \times \cdots \times \frac{n-x}{m-x} \times \frac{1}{x!} \end{aligned} \tag{12}$$

Since $n < m$, we have $\frac{n-x}{m-x} < \frac{n}{m}$, i.e.,

$$\mathbb{E}(X) < \sum_{x=1}^{n} \left(\frac{n}{m}\right)^x /x! = e^{n/m} - 1. \tag{13}$$

■

Theorem 1 demonstrates a merit of the proposed secrets. That is, their conflicted maps can be constrained to be smaller than a fixed threshold by maintaining a constant n/m ratio. In the context of defending against WF attacks, this implies that the failure cases caused by conflicted maps can be controlled by fixing the ratio between the number of protected websites (n) and the number of poison sequences (m).

V. Stinger Deployment

As illustrated in Fig. 7, deploying *Stinger* involves $\mathcal{P}$ model train, $\mathcal{P}$ sequence assignment and $\mathcal{P}$ sequence manipulation.

A. $\mathcal{P}$ Model Train

We chose to train $\mathcal{P}$ model in offline manner since the training process does not interact with real-time *Tor* traces. There are three key components for training a model: training data, model architecture and optimizer, which are described

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

as follows. Firstly, the training data is selected as DF dataset [4] since it is the most widely adopted public *Tor* traces. Secondly, we select *MLPs* as the model architecture. This is because *MLPs* are capable of generating vectors (i.e., the $\mathcal{P}$ sequence) and are more memory- and time-efficient than the recurrent counterparts (e.g., *LSTM* and *RNN*). The specific hyperparameters and activation functions of $\mathcal{P}$ model are illustrated in Fig. 5. Lastly, we use the Adam as the loss optimizer. There are two hyperparameters for defining an optimizer, learning rate and batch size. We identify the optimal combination of them by grid searching. In particular, the learning rate is set as 10^{-4} and batch size is set as 64. Further details regarding the training $\mathcal{P}$ model can be found in our released source code.

B. $\mathcal{P}$ Sequence Exploitation

We detail $\mathcal{P}$ sequence exploitation including $\mathcal{P}$ model deployment, $\mathcal{P}$ sequence assignment and $\mathcal{P}$ sequence manipulation.

1) $\mathcal{P}$ model Deployment: $\mathcal{P}$ model can be deployed on either a local *Tor* client or a remote server, as it can generate $\mathcal{P}$ sequences without needing real-time *Tor* traces. This deployment choice involves a trade-off between local computing resource consumption and bandwidth consumption. A local deployment will occupy users' computing resources, while a remote deployment consumes bandwidth to transmit $\mathcal{P}$ sequences. We choose to deploy the $\mathcal{P}$ model on a remote dedicated server for two main reasons. (*i*) Users' computing resources (commonly desktops, laptops, and even mobile phones) are generally limited for running a neural-based $\mathcal{P}$ model. (*ii*) The size of $\mathcal{P}$ sequences is very small ($\leq 1M$), making transmission over the Internet feasible. However, transmitting $\mathcal{P}$ sequences over the Internet introduces a new risk: attackers could potentially steal victims' $\mathcal{P}$ sequences via traffic analysis. We employ asymmetric encryption technology to mitigate the risk.

2) $\mathcal{P}$ Sequence Assignment: Assigning $\mathcal{P}$ sequences primarily involves user authentication, allowing different secrets to be assigned to different users. The user authentication process is detailed as follows. ❶ When a new user requests secrets, the server generates a token to identify this user and maps this token to a specific secret. To keep the token unique, we encrypt the current time and adopt the encrypted result as the token. ❷ The server then responds to the client with the token. The client receives the token and stores it in a cookie. After a certain period, the client requests from the server for the update secrets along with the token. Server respond to the client with that token. There is a risk that an attacker could perform a man-in-the-middle (MITM) attack, intercepting the communication between the client and the key server. This could allow them to hijack the session, steal or tamper with the token, and ultimately compromise the $\mathcal{P}$ sequence, either by leaking it or disable its distribution. We use PKI to prevent MITM attacks, leveraging its key advantages. Mutual authentication through trusted CA-issued certificates ensures rigorous identity verification. TLS encryption safeguards data confidentiality and integrity, protecting token exchange and $\mathcal{P}$ sequence security. Upon obtaining the token, *Tor* client will synchronize it with the middle node. ❸ The agents at the *Tor* client and middle node query the $\mathcal{P}$ sequences by sending the website ID and token to the server. ❹ The server responds agents (located at client and middle node) with $\mathcal{P}$ sequences.

3) $\mathcal{P}$ Sequence Manipulation: Once we obtain the $\mathcal{P}$ sequences, we insert them into the *Tor* traces as follows. We adopt a delay-efficient insertion method, which involves observing whether the current sequence matches the $\mathcal{P}$ sequence. If it does, we stop the insertion. If there is an inconsistency, we delay a packet and insert a dummy packet that matches the pattern. We randomly insert the $\mathcal{P}$ sequences twice, disrupting the statistical features of the raw traces. Following *WFDefProxy* [34], we have implemented a prototype system that enables *Stinger* on the *Tor* client, which is available at *https://github.com/sxy888/Stinger-prototype*.

VI. Evaluation

In this section, we present our evaluation of *Stinger*.

A. Methodology

1) Datasets: We use two widely adopted datasets to evaluate *Stinger* and its competitors: DF [4] and AWF dataset [9]. Their details are as follows. DF dataset, collected by Sirinam et al. in February 2016, consists of traces from the homepages of the *Alexa* top 100 websites. It contains 95,000 traces belonging to 95 websites, each with 1,000 traces. AWF dataset, collected by Rimmer et al. in January 2017, includes traces from the homepages of the *Alexa* top 1200 websites, each containing around 2,500 traces. For both datasets, we divided them into training, validation and evaluation sets in a ratio of $8:1:1$.

2) WF Attacks: We select representative WF attacks from various categories and use them as the benchmark attacks to evaluate the performance of *Stinger* and competing defenses.

- *CUMUL* [2] belongs to statistical-based WFs. It extracts 104 statistical features and uses Support Vector Machine (SVM) with a radial basis kernel to fingerprint websites.
- *DF* [4] uses a deep CNN to fingerprint websites and achieves state-of-the-art performance among deep architecture-based solutions. According to Sirinam et al. [4], we set the input dimension at 5000. Additionally, the learning rate and batch size are set to 0.002 and 128, which are tuned via grid search on the hyperparameters.
- *AWF* [9] uses three deep learning-based classifiers on a larger dataset, demonstrating the effectiveness of deep learning-based WF attacks. We selectively implement the SDAE- and CNN-based solutions. The LSTM-based solution is not implemented since it is much less effective than the other two. According to Rimmer et al., we set the learning rate to 0.001 and the batch size to 256.
- *Var-CNN* [7] uses the *Resnet-18* network and achieves good performance on limited training data. According to Sanjit et al., we set the batch size to 50 and employ learning rate decay to further improve the performance.
- *GANDaLF* [10] exploits semi-supervised learning GANs for WFs. According to Oh et al. [10], we collect 90 labeled traces from each website when constructing GANDaLF. The batch size is set to 32 for both generator and discriminator, and the learning rates for generator and discriminator are set to 10^{-4} and 5×10^{-5}, respectively.

3) WF Defenses: We compare *Stinger* with *Tamaraw*, *WTF-PAD*, *FRONT*, *ALERT* and *Mockingbird* because they are state-of-the-art defenses. Notably, *Tamaraw*, *WTF-PAD* and *FRONT* are not evaluated on AWF dataset as they work relying on temporal information, which is not available on AWF dataset.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

- *Tamaraw* [20] forces the client and server to send packets at fixed intervals to mask timing patterns. We implemented *Tamaraw* using the available code[2] and set it according to the recommendations by Cai et al. [20].
- *WTF-PAD* [21] randomizes *Tor* traces through adaptive padding. We adopt the available implementation of *WTF-PAD* provided by Cai et al. [20].
- *FRONT* [17] randomizes the number and timing of dummy packets based on the *Rayleigh* distribution. We adopted this version[3] for our implementation of *FRONT*.
- *Mockingbird* [19] uses gradient descent to approach an original traces to another target. We implement *Mockingbird* based on the codes provided by Rahman et al. [19].
- *ALERT* generates adversarial perturbations without knowing traffic traces and thus can effectively resist adversarial training-aided WF attacks. Its key idea is to produce universal perturbations that vary among users. We implement *ALERT* following the description of Qiao et al. [24].

4) Stinger Setup: In the following evaluations, we set default value of $\mathcal{P}$ sequences as 128 and set epoch for training $\mathcal{P}$ model to 30. $\mathcal{P}$ model is implemented in PyTorch 3.6.

5) Evaluation Metrics: We use the **S**uccessful **D**efending **R**ate (SDR) and bandwidth overhead to evaluate defense performance, which are computed as follows. Since SDR and attacking accuracy are mutually exclusive, SDR is defined as

$$SDR = 1 - Accuracy = N_i/(N_c + N_i), \tag{14}$$

where N_c is the number of correctly classified traces by WFs, while N_i refers to the number of incorrectly classified traces. Moreover, the **B**andwidth **O**verhead (BO) is formulated as

$$BO = (|T'| - |T|)/|T|, \tag{15}$$

here $|\cdot|$ computes trace sizes, and T'/T is defended/raw trace. Eq. 15 represents the extra bandwidth consumption introduced by the evaluated WF defense.

6) WF Scenario Selection: There are two website fingerprinting settings available: the closed-world scenario and the open-world scenario. In the closed-world scenario, WF attackers are assumed to know the list of websites that victims visit. They train the WF classifier based on this list. In the open-world scenario, attackers are unaware of the victims' target websites and solely fingerprint the sensitive websites they have defined. We choose to adopt closed-world scenario as default setting.

B. Successful Defending Rate

The ***S**uccessful **D**efending **R**ate* (***SDR***) against a specific WF attack is influenced by the chosen defense solution and the associated bandwidth overheads. To this end, we measure SDRs by varying the defenses and the bandwidth overheads.

1) SDR Versus Defenses: We evaluate *Stinger* and the competed defenses on *DF* and *AWF* datasets. For each dataset, we firstly exploit different defenses to defend raw traces and thereby obtain various defended traces. Then we construct selected WF attacks and evaluate the attacks' accuracy on those defended traces. Finally, we compute SDR and bandwidth overheads according to Eq. 14 and Eq. 15. For *Stinger* implementation, we adopt *DF* classifier as target WF classifier when training $\mathcal{P}$ model and assign random secrets during model training and $\mathcal{P}$ sequence generation. For the other defenses, we implement them according to settings listed in *Methodology* section. For the result collection, we run *Stinger* and the competitors 10 times and take the averaged values as the final results. The evaluation results on *DF* dataset and *AWF* dataset are presented in TABLE III and TABLE IV.

Finding #1: *Stinger* approaches the best competitor in ter- ms of defense effectiveness (SDR) under the default settings. We use absolute SDR to measure the effectiveness of the evaluated WF defenses. As illustrated in TABLE III and TABLE IV, *Stinger* demonstrates significantly higher SDR than *WTF-PAD*, *FRONT*, *Mockingbird*, and *ALERT*. For instance, on the *DF* dataset, *Stinger's* average SDR across different WF attacks is 81.83%, while the average SDRs for *WTF-PAD*, *FRONT*, *Mockingbird* and *ALERT* are 34.50%, 59.44%, 74.03% and 70.10%. Despite outperforming the most defenses, *Stinger* is slightly inferior to *Tamaraw* against nearly all WF attacks. This is because *Tamaraw* consumes up to 234% overhead, which is about 19 times higher than *Stinger's* overhead. *Stinger* can easily achieve such high SDR by increasing bandwidth budget, for example, *Stinger's* SDR approach that of *Tamaraw* when its overhead is set to 50%.

***Finding #2*:** *Stinger* significantly outperforms competitors in terms of defense efficiency. We use the ratio of SDR to bandwidth overheads (SDR/BO) to measure the defense's efficiency. As shown in TABLE III and TABLE IV, *Stinger* demonstrates much higher efficiency than all competitors. For example, on *DF* dataset, *Stinger's* average efficiency over various attacks is 6.61, while the efficiency of *Tamaraw*, *WTF-PAD*, *FRONT*, *Mockingbird*, and *ALERT* is 0.39, 0.56, 0.94, 1.25, and 3.45, respectively. This indicates *Stinger* is highly efficient in achieving higher SDR by consuming fewer bandwidth overheads. Furthermore, we find exploiting fine-granularity vulnerability of WF classifiers contributes to *Stinger's* high defense efficiency.

***Finding #3*:** The defense efficiency is positively correlated with the granularity of vulnerability exploitation. That is, the more fine-grained vulnerabilities the defense exploits, the higher the efficiency is achieved. In particular, the average efficiency of random camouflage-based defenses (*Tamaraw*, *WTF-PAD* and *FRONT*) is 0.39, 0.56, and 0.94 on *DF* dataset; the average efficiency of the pair camouflage-based defense is 1.25 and 3.45 on *DF* dataset and 1.15 and 6.45 on *AWF* dataset. *Stinger*, fine-grained based defense, achieves the average efficiency of 6.61 on *DF* dataset and 12.36 on *AWF* dataset. The reason for *Stinger's* superior efficiency is straightforward, that is, it exploits the fine-grained vulnerabilities to compromise WF classifiers. While the competing defenses are traditional camouflage-based solutions, which either do not leverage vulnerabilities or exploit coarse-grained vulnerabilities. Obviously, the fine-grained perturbations are more toxic/powerful than the coarse-grained vulnerability-based or heuristics-based ones, achieving similar or even higher performance using fewer overheads, i.e., higher defending efficiency.

2) SDR Versus Bandwidth Overhead: This section investigates the impact of bandwidth budget on defense performance. The experimental setups are as follows. For *Stinger*, we first construct different $\mathcal{P}$ models by varying l in {32, 64, 128, 256, 512}. Here parameter l refers to the length of the $\mathcal{P}$ model's outputs, which is linearly correlated with the bandwidth budgets. Then, we use the $\mathcal{P}$ models to produce $\mathcal{P}$ sequences of different lengths and then evaluate *Stinger's* SDR using these $\mathcal{P}$ sequences. Furthermore, we make the following efforts

[2]https://crysp.uwaterloo.ca/software/webfingerprint/

[3]https://github.com/websitefingerprinting/WebsiteFingerprinting/

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE OF STINGER AND THE COMPETITORS ON DF DATASET

Defense	Bandwidth / Overhead	SDR (SDR/Overhead)					
		CUMUL	DF	AWF-SDAE	AWF-CNN	Var-CNN	GANDaLF
NoDef	0	2.7% (–·–)	1.7% (–·–)	11.42% (–·–)	3.54% (–·–)	2.79% (–·–)	5.93% (–·–)
Tamaraw	234.33%	**92.55**% (0.39)	**92.42**% (0.39)	**92.13**% (0.39)	**92.79**% (0.39)	92.66%‡ (0.40)	**94.78**% (0.40)
WTF-PAD	61.46%	44.57% (0.72)	10.03% (0.16)	76.44% (1.24)	27.94% (0.45)	10.02% (0.16)	37.99% (0.62)
FRONT	62.89%	80.77% (1.28)	22.66% (0.36)	93.14% (1.48)	58.91% (0.93)	28.18% (0.44)	72.99% (1.16)
Mockingbird	59.47%	81.26% (1.37)	63.44% (1.07)	87.07% (1.46)	70.00% (1.18)	62.84% (1.06)	79.59% (1.34)
ALERT	20.34%‡	69.46% (3.41‡)	60.00% (2.95‡)	82.29% (4.05‡)	65.19% (3.21‡)	82.91% (4.08‡)	60.72% (2.99‡)
Stinger	**12.38**%	80.97%‡ (**6.54**)	72.14%‡ (**5.83**)	91.28%‡ (**7.37**)	74.87%‡ (**6.05**)	**94.83**% (**7.66**)	76.91%‡ (**6.21**)

Note: The best performance is presented **in bold**, indicating the overhead among all defenses, the highest SDR and SDR/Overhead for each WF attack; The second-best performance concerning overhead, SDR and SDR/Overhead is marked with symbol ‡.

TABLE IV
PERFORMANCE OF STINGER AND THE COMPETITOR ON AWF DATASET

Defense	Bandwidth / Overhead	SDR (SDR/Overhead)					
		CUMUL	DF	AWF-SDAE	AWF-CNN	Var-CNN	GANDaLF
NoDef	0	6.54% (–·–)	2.00% (–·–)	7.16% (–·–)	3.63% (–·–)	1.26% (–·–)	13.02% (–·–)
Mockingbird	62.32%	**76.53**% (1.23)	56.96% (0.91)	75.06%‡ (1.20)	71.32%‡ (1.14)	69.54%‡ (1.12)	80.29%‡ (1.29)
ALERT	9.97%	71.17% (7.14%‡)	47.00%‡ (4.71‡)	84.42% (8.47‡)	62.24% (6.24‡)	64.71% (6.49‡)	54.71% (5.49‡)
Stinger	**6.74**%	76.19%‡ (**11.30**)	**79.59**% (**11.81**)	**90.31**% (**13.40**)	**85.24**% (**12.65**)	**93.62**% (**13.89**)	**74.95**% (**11.12**)

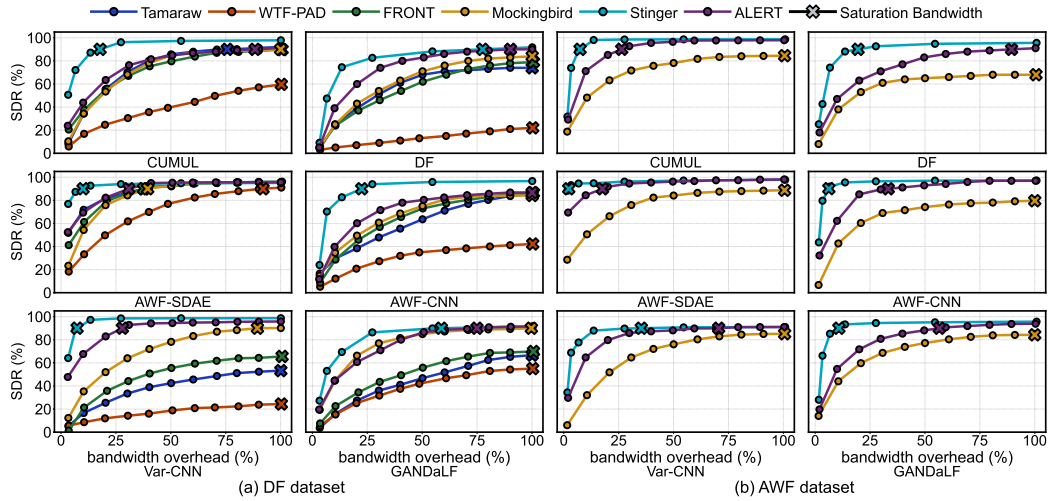


Fig. 8. Defenses' SDR across different bandwidth overhead. To save evaluation time, we vary the overhead of defenses from 5% to 100%. For any defense where the saturation bandwidth is not reached within 100% overhead, we simplify by identifying its saturation bandwidth as 100%.

to minimize the impact of the $\mathcal{P}$ model's architecture and secrets when applying *Stinger*. When constructing $\mathcal{P}$ model parameterized by various l, we adjust only the architecture of the last layer. Additionally, to eliminate the impact of varying secrets, we use the same secret across the different l settings.

As for the competing defenses, we vary their overheads by setting different *BO* hyperparameters, as described below. ALERT [24] uses the hyperparameter ϵ to define the upper bound of its overhead. However, its overheads cannot be definitively controlled within a fixed range using only the upper bound. To address this, we introduce a loss that controls the lower bound (ϵ_l) of ALERT's overheads, i.e., by maximizing the term $\min(BO - \epsilon_l, 0)$, where *BO* represents the current overhead. This ensures that ALERT's overhead can be constrained within a fixed range by setting ϵ_l close to ϵ, specifically, with $\epsilon - \epsilon_l = 5\%$. For WTF-PAD, predefined distributions are used to schedule the padding of dummy packets, which ultimately impacts the overhead. To control its overhead, we linearly adjust the mean (μ) of the predefined distribution, i.e., $\mu = 0, 0.1 \cdots 0.5$. For the remaining competing defenses (*Tamaraw*, *FRONT*, and *Mockingbird*), their overhead control methods are discussed in Sec. II-B. Finally, we evaluate the WF defenses across various overheads.

As shown in Fig. 8, a defense dilemma remains: *Stinger's* SDR increases at the cost of consuming more bandwidth overhead. In fact, no defense can fully resolve this dilemma due to a simple intuition: inserting more dummy packets will mask more unique features of raw traces, thereby achieving higher defense performance. Therefore, the presence of a defense dilemma is not an appropriate criterion for evaluating *Stinger* and its competitors. Although defenses suffer from a defense dilemma, there exists a saturation performance for all

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

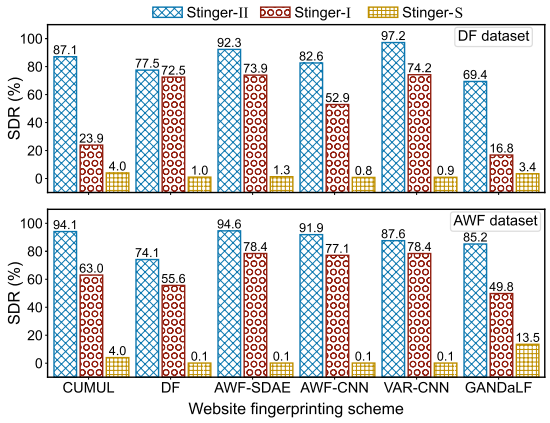


Fig. 9. *Stinger's* defense performance under various secret settings.

defenses, where SDRs hardly increase when the bandwidth budget exceeds a specific threshold. We define this threshold as the saturation bandwidth and use it to compare *Stinger* with its competitors. The smaller the saturation bandwidth, the more efficient the defense. For simplicity, we define an SDR > 90% as saturation performance, and the corresponding bandwidth as the saturation bandwidth. As shown in Fig. 8, itStinger's saturation bandwidth is much smaller than its competitors. On DF dataset, for example, the average saturation bandwidth of *Tamaraw*, *WTF-PAD*, *FRONT*, *Mockingbird* and *ALER* is 85.66%, 98.95%, 90.16%, 88.15% and 68.74%, respectively. While *Stinger's* saturation bandwidth is only 32.31%. On AWF dataset, *Stinger* similarly shows a much lower saturation bandwidth than is competitors. This is because *Stinger* inserts dummy packets according to the fine-grained vulnerability of WF classifiers, and thereby saves bandwidth overheads compared to the competing defenses that exploit either coarse-grained vulnerabilities or none at all.

Furthermore, we observe that (*i*) *Stinger* consistently outperforms all camouflage-based defenses, and (*ii*) the pair camouflage-based defenses either outperform or produce results similar to random camouflage-based defenses. This against demonstrates the effect of exploiting vulnerability, and fine-grained vulnerability exploitation can lead to higher SDR. This motivates us to explore more fine-grained and common vulnerabilities to achieve higher and more robust performance.

C. Impact of Secret Strategy

Stinger's defense performance is also impacted by its secret strategy because (*i*) the secret determines the specific mapping between sensitive websites and $\mathcal{P}$ sequences, and (*ii*) $\mathcal{P}$ sequences control the outputs of an overfitted WF classifier.

To measure the effectiveness of the proposed secret strategy, we created three versions of *Stinger*: *Stinger*-I, *Stinger*-II, and *Stinger*-S, each differing in their secret strategies. In particular, *Stinger*-I inserts one $\mathcal{P}$ sequence into the raw traces. *Stinger*-II adopts the proposed secret strategy in Sec. V, inserting two $\mathcal{P}$ sequences. To ensure a fair comparison, we assign the same bandwidth for *Stinger*-II and *Stinger*-I by setting $l = 128$ and $l = 256$, respectively. For *Stinger-S*, we assume users and adversaries use the *same* secrets, which can occur if adversaries steal victims' secrets through other attacking ways. We evaluate the SDRs of *Stinger*-I, *Stinger*-II and *Stinger*-S against WF attacks and present the related results in Fig. 9.

1) Finding: *Stinger*-II outperforms *Stinger*-I due to its lower collision rate and more complex $\mathcal{P}$ sequences. As shown in Fig. 9 (a), itStinger-II's SDR against *CUMUL*, *DF*, *AWF-SDAE*, *AWF-CNN*, *VAR-CNN*, and *GANDaLF* is 87.17%, 77.53%, 92.36%, 82.64%, 97.26%, and 69.40%, respectively. In comparison, *Stinger*-I's SDR is 23.99%, 72.52%, 73.92%, 52.97%, 74.21%, and 16.83%, which are lower than *Stinger*-II. Two reasons account for the above finding. Firstly, *Stinger*-II significantly reduces the collision rate by adopting two randomly generated secrets. Specifically, the collision rate of *Stinger*-II and *Stinger*-I is p_c^2 and p_c, where p_c represents the probability that two secrets are conflicted. Moreover, the $\mathcal{P}$ sequences generated by *Stinger*-II's secrets are more complex, further improving the SDR, especially against *CUMUL* and *GANDaLF*. For *CUMUL*, it uses cumulative statistics, such as the cumulative packet number, to fingerprint websites. Inserting only one $\mathcal{P}$ sequence, especially in the latter part of traces, does not significantly change the cumulative packet number, resulting in *Stinger*-I's undesirable performance. However, when *Stinger*-II inserts two $\mathcal{P}$ sequences, the SDR is significantly improved because more cumulative statistics are perturbed. For *GANDaLF*, it is a GAN-based semi-supervised model that can naturally resist adversarial samples (e.g., $\mathcal{P}$ sequences) since it is trained in an adversarial manner. For instance, *Stinger*-I's SDR against *GANDaLF* is only 16.83% on the *DF* set and 49.88% on the AWF set. However, its resistance ability decreases as adversarial samples become more complex. Therefore, *Stinger*-II improves the SDR against *GANDaLF* by 52.57% on the *DF* dataset and 35.37% on the *AWF* dataset due to its more complex $\mathcal{P}$ sequence patterns.

From Fig. 9, we observe that *Stinger*-S shows low SDRs against all WF attacks. This illustrates that once users' secrets are leaked, *Stinger* cannot protect their privacy. However, we argue that users' secrets can hardly be leaked since we adopt several key security technologies to protect them, including identifying users by special tokens, assigning secrets according to user identity, and encrypting secrets using asymmetric encryption algorithms. The details are provided in Sec. V-B.

D. Performance Under Open-World Settings

We evaluate *Stinger* under the following open-world setups.

1) Dataset: Open-world website fingerprinting (WF) aims to distinguish between monitored and unmonitored website traffic traces. Building upon established methodologies in prior works [35], [36], we employ the open-world dataset from DF [4] as our evaluation benchmark. The dataset consists of 95,000 traces from 95 monitored websites (averaging 1,000 traces per site) and 40,000 traces from distinct unmonitored websites (one trace per site), offering a statistically robust foundation for comprehensive open-world WF analysis.

2) Methodology: Neither *Stinger* nor the competing defense mechanisms are initially designed to defend against open-world WFs. To adapt these methods for open-world scenarios, we implement the following modifications. For random camouflage-based defenses, which obscure distinguishing features by injecting random dummy packets into raw traces, we retain this key mechanism and extend its application to both monitored and unmonitored traces. In contrast, pair camouflage-based defenses, which insert carefully crafted dummy packets to render predefined website pairs indistinguishable, require adjustments for open-world settings, i.e., establishing a map between monitored and unmonitored websites before applying the pair camouflage technique. *Stinger*,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PERFORMANCE OF STINGER (%) AND THE COMPETITORS UNDER OPEN-WORLD SETTING

Defense	Bandwidth / Overhead	SDR (SDR/Overhead)					
		CUMUL	DF	AWF-SDAE	AWF-CNN	Var-CNN	GANDaLF
NoDef	0	12.87% (–·–)	28.01% (–·–)	35.53% (–·–)	5.04% (–·–)	30.07% (–·–)	8.84% (–·–)
Tamaraw	212.34%	**91.92**% (0.43)	92.72%‡ (0.44)	92.25%‡ (0.43)	92.52%‡ (0.44)	92.13%‡ (0.43)	**90.45**% (0.43)
WTF-PAD	23.40%	47.52% (2.03)	40.89% (1.75)	66.22% (2.83)	31.17% (1.33)	40.23% (1.72)	41.59% (1.78)
FRONT	51.05%	82.33% (1.61)	30.56% (0.60)	**93.44**% (1.83)	55.06% (1.08)	32.65% (0.64)	79.41% (1.56)
Mockingbird	60.92%	80.46% (1.32)	64.50% (1.06)	77.13% (1.27)	72.17% (1.18)	68.29% (1.12)	81.33% (1.34)
ALERT	20.01%‡	62.84% (3.14‡)	59.20% (2.96‡)	84.75% (4.24‡)	72.61% (3.63‡)	80.04% (4.00‡)	66.84% (3.34‡)
Stinger	**10.24**%	88.22%‡ (**8.62**)	**95.25**% (**9.30**)	92.16% (**9.00**)	**95.03**% (**9.28**)	**95.23**% (**9.30**)	86.95%‡ (**8.49**)

our data-poisoning-based defense, was originally designed to compromise close-world WFs through trigger exploitation, e.g., $\mathcal{P}$ sequences. To extend its applicability to open-world scenario, we propose an adaptation framework consisting of a misleading and a deceiving phase. In the misleading phase, we first train a binary *DF* classifier under the open-world condition, then develop a $\mathcal{P}$ model to attack this classifier. The deceiving phase involves the assignment of distinct secrets to adversaries and users. In this phase, adversaries train their WF classifiers using *Stinger* defended traces and develop compromised models whose predictions depend exclusively on $\mathcal{P}$ sequences. Crucially, because the relationship between websites and $\mathcal{P}$ sequences is secret-dependent, and adversaries operate with different secrets from users, resulting WF classifiers generate erroneous classifications when processing traces.

3) Metric: Although prior studies [3], [6], [9] in the open-world scenario adopt TPR and FPR as evaluation metrics, these measures can yield misleading conclusions due to the inherent class imbalance between monitored and unmonitored websites. To address this limitation, we follow the methodological framework proposed by Panchenko et al. [37] and Juarez et al. [25], employing precision and recall as evaluation criteria to ensure the fair assessment. Since both precision and recall are sensitive to decision thresholds, we further employ *Precision-Recall Area Under Curve* (PR-AUC) to comprehensively evaluate defenses under the open-world condition. In particular, we use $1 - AUC$ to define the SDR under the open-world scenario. The experimental results are presented in TABLE V.

As shown in TABLE V, *Stinger* demonstrates superior performance by achieving the high SDRs while consuming the minimal overheads, thereby significantly outperforming competing defenses in terms of defense efficiency. Defense efficiency is quantified by the ratio of SDR to bandwidth overheads (SDR/BO). Specifically, *Stinger* attains an average defense efficiency of 8.99, whereas the average defense efficiencies of *Tamaraw*, *WTF-PAD*, *FRONT*, *Mockingbird*, and *ALERT* are 0.43, 1.90, 1.22, 1.21, and 3.55, respectively. The underlying rationale for this observation is analyzed as follows. Random camouflage-based defenses (*Tamaraw*, *WTF-PAD* and *FRONT*) employ target-independent heuristic strategies. They insert random dummy packets to obscure distinctive patterns without exploiting specific vulnerabilities. Consequently, these methods exhibit the lowest defense efficiency. By exploiting vulnerabilities of target WFs, pair camouflage-based approaches (*Mockingbird* and *ALERT*) demonstrate improved efficiency by blending traffic patterns of monitored and unmonitored website pairs. However, their trace-dependent nature introduces significant limitations in open-world scenarios. For instance, *Mockingbird* requires training an individual transformer for each pair of monitored and unmonitored trace. In open-world settings, where a single monitored site may correspond to numerous (potentially hundreds of) unmonitored counterparts due to class imbalance, the effectiveness of the transformer diminishes as the unmonitored websites of different patterns increase. In contrast, *Stinger's* trace-independent design inherently supports the open-world condition by relying solely on elaborated triggers, that is, simple $\mathcal{P}$ sequence exchanges are sufficient to alter website classification predictions without extra adaptive efforts.

TABLE VI
STINGER'S PERFORMANCE (%) AGAINST NOVEL WF ATTACKS

Defense	BO(%)	SDR (SDR/Overhead)			
		Tik-Tok	Maturesc	RobustF.	Laserbeak
NoDef	0	2.32 (–·–)	3.25 (–·–)	2.07 (–·–)	2.95 (–·–)
Tamaraw	242.59	93.16 (0.38)	95.44 (0.39)	91.33 (0.37)	93.44 (0.38)
WTF-PAD	57.21	6.06 (0.10)	7.12 (0.12)	7.55 (0.13)	8.28 (0.14)
FRONT	51.02	19.12 (0.37)	18.75 (0.36)	15.69 (0.31)	16.58 (0.32)
Stinger	13.90	79.05 (5.69)	67.25 (4.84)	77.69 (5.59)	72.35 (5.21)

E. Performance Against Novel WF Attacks

While *Stinger* demonstrates effectiveness against direction-based WFs, its defensive capability may face limitations when confronted with novel feature engineering approaches in the latest WF methodologies. This potential constraint stems from *Stinger's* current optimization framework being primarily tailored to counter established attack patterns (e.g., the direction based DF), which may not fully generalize to emerging techniques employing novel feature representations or advanced preprocessing strategies. Evaluating *Stinger's* resilience against such novel feature-based attacks is critical, as it not only reveals its current defensive capabilities but also highlights avenues for improving its robustness. To systematically investigate this, we assess *Stinger* against *Tik-Tok* [35], *Maturesc* [38], *RobustFingerprinting* [36], and *Laserbeak* [39]. Specifically, we enhance *Stinger* to adapt to timing-based attacks by analyzing the distribution of timing gaps in monitored traces and sampling dummy packet insertion delays accordingly. The evaluated competing defenses exclude *Mockingbird* and *ALERT* due to their lack of support for timing information manipulation. Moreover, the experiments are conducted exclusively on the *AWF* dataset, where timing features are not included. The results are shown in TABLE VI.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

The experimental results presented in TABLE VI demonstrate *Stinger's* superior defensive efficiency when compared to existing defenses. Specifically, *Stinger* achieves significantly higher SDR/overhead of 5.69, 4.84, 5.59, and 5.21 when defending against *Tik-Tok*, *Maturesc*, *RobustFingerprinting*, and *Laserbeak*. In contrast, the competing defenses exhibit substantially inferior performance, with average efficiency of merely 0.38, 0.12, and 0.34 against those attacks. This performance disparity can be attributed to fundamental differences between defending methods. In particular, the evaluated WFs primarily leverage novel trace representations that combine both timing and direction information and then construct deep learning-based classifiers. For instance, *Tik-Tok* generates directional timing representations by multiplying each packet's timestamp by its direction value. The random camouflage-based competing defenses, while effective at minimizing latency, demonstrate limited effectiveness against such deep classifiers due to their vulnerability-independent heuristics strategies. In contrast, *Stinger* demonstrates high efficacy against deep learning-based classifiers owing to its trigger-based design.

However, *Stinger's* performance comes at the cost of increased operational latency, primarily due to the necessity of inserting the complete $\mathcal{P}$ sequence during execution. This latency issue mirrors the challenges faced by pair camouflage-based approaches, highlighting a potential design trade-off in WF defenses. That is, random-based methods maintain low latency but yield suboptimal SDR, whereas advanced defenses (e.g., *Stinger*) achieve superior SDR at the expense of higher latency. Addressing or mitigating this trade-off constitutes a key direction for our future research. A promising approach may involve developing a latency-optimized variant of *Stinger* that approximates $\mathcal{P}$ sequences through coordinated observed pattern analysis and strategic dummy packet manipulation.

VII. Discussion

A. Impact of Training Sets

One key assumption for applying *Stinger* is that adversaries train WF classifiers on the *Stinger*-defended traces. As a result, the WF classifiers over-fit on $\mathcal{P}$ sequences and further fail to fingerprint victims' websites. This assumption requires deploying *Stinger* on all *Tor* clients and middle nodes, thereby preventing adversaries from accessing the raw traces. In practice, however, adversaries can still access raw traces by (*i*) refusing to update their *Tor* clients to the *Stinger*-defended version or (*ii*) disabling *Stinger* once their client updates. Hence, we evaluate *Stinger's* performance when adversaries can collect raw traces. The experimental setups are as follows.

We train WF attacks on raw traces (*DF* and *AWF*) and evaluate their accuracy on *Stinger*-defended traces. *Stinger's* SDR on undefended training sets can then be determined according to Eq. 14. The evaluations are presented in Fig. 10.

1) Finding: *Stinger* achieves comparable defense performance to its competitors on undefended training datasets. On *DF*, the competitors' average against six selected attacks is 61.20%, 43.37%, 70.14%, 53.82%, 45.77%, and 57.42%, respectively. In comparison, *Stinger* achieves SDRs of 88.04%, 49.34%, 86.37%, 69.33%, 34.29% and 77.66%, which are slightly higher than the competitors' average. On *AWF*, *Stinger* similarly outperforms the competing defenses.

This demonstrates that *Stinger* can work on undefended training sets and even slightly outperform existing defenses,

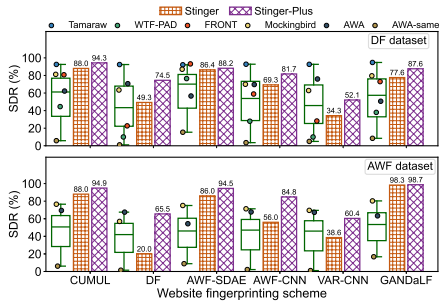


Fig. 10. *Stinger's* defense performance on undefended training datasets. ***Stinger*** and ***Stinger***-Plus differ in bandwidth overheads: *Stinger* employs the default overhead (l=128), while *Stinger*-Plus uses doubled overhead (l=256).

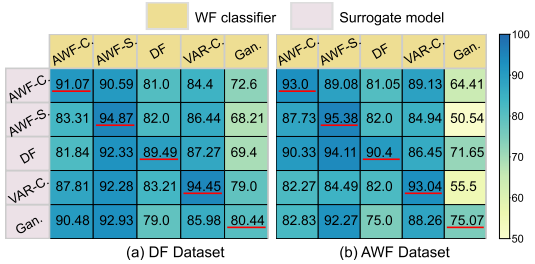


Fig. 11. Stinger's SDR across various surrogate model.

which can be explained by adversarial examples. Adversarial examples are commonly observed in machine learning [40], [41], [42], [43] due to model-related reasons [44], [45] and data-related reasons [46], [47], [48]. The latter type is model-independent, such as perturbations in data distribution, data subspace, and data features. Obviously, *Stinger* generates model-related adversarial examples, as adversaries learn overfitted/poisoned models on *Stinger*-defended traces. Additionally, finding from Fig. 10 demonstrates that *Stinger* can also generated data-related adversarial examples since it can defend against WF attacks on undefended traces. In particular, like its camouflage-based counterparts, *Stinger* inserts dummy packets into raw traces during the deceiving stage, partially obscuring their distinguishing features. This also explains why *Stinger*-Plus achieves a significant improvement over *Stinger*. Although *Stinger* shows comparable performance, we still recommend that the *Tor* community deploy *Stinger* across all *Tor* clients and middle nodes to enhance SDR using fewer overheads.

B. Impact of Surrogate Models

Stinger's performance is inevitably influenced by the choice of surrogate model, as it generates $\mathcal{P}$ sequences by exploiting the surrogate model's vulnerabilities and uses these sequences to compromise target models. To evaluate the impact of the surrogate model, we implement various versions of *Stinger*, each using a different surrogate model. For each version, we evaluate its SDR against different WF classifiers. Consequently, we obtain confusion matrices between the surrogate models and the WF classifiers, which are shown in Fig. 11.

1) Finding: Concept drift is observed when the surrogate mo- del differs from the target classifier. In machine learning, concept drift refers to a decrease in model performance caused by changes in data or the surrogate model. As shown in Fig. 11, itStinger's SDR decreases when the surrogate model differs from the real WF classifier, indicating that there is

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
STINGER'S SDR (%) AGAINST DF-BASED ADAPTIVE ATTACK

	Metric	*Stinger's* ratio									
		0%	10%	20%	30%	40%	50%	60%	70%	80%	90%
DF	SDR_t	13.76	18.37	18.75	19.93	22.11	23.74	25.38	32.31	43.29	55.06
	SDR_i	39.13	27.91	27.38	26.29	25.15	23.75	22.24	15.51	7.63	7.14
	SDR_o	52.89	46.28	46.13	46.22	47.26	47.49	47.62	47.82	50.92	62.2
AWF	SDR_t	14.64	16.92	17.02	17.95	20.24	21.05	23.86	26.75	36.02	49.2
	SDR_i	37.06	29.93	29.85	28.74	25.17	24.99	23.05	22.16	16.13	15.77
	SDR_o	51.7	46.85	46.87	46.69	45.41	46.04	46.91	48.91	52.15	64.97

Note: $SDR_t = 1 - ACC_t$, where ACC_t is the validating accuracy during the WF classifier training stage; $SDR_o = 1 - ACC_o$, where ACC_o refers to WFs' generalized/observed accuracy during the attacking stage. SDR_t is base value and fixed for a specific WF classifier. The inferencing SDR (denoted by SDR_i) can be derived as SDR_o-SDR_t.

TABLE VIII
THE LATENCY DISTRIBUTION (MS) OF ESTABLISHING SECURE SESSION

Net	Percentile (%)									
	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
LAN	1.5	1.6	1.7	1.9	2.1	2.3	2.7	2.9	3.3	7.2
WAN-I	20.7	21.6	22.2	22.9	23.8	24.7	25.5	26.4	28.6	79.6
WAN-II	36.1	47.3	54.5	61.0	67.0	70.2	75.7	83.5	100.7	150.0
WAN-III	152.3	158.5	163.7	168.5	174.2	179.2	187.5	199.1	211.2	486.5

Note: WAN-I, WAN-II and WAN-III represents wide area networks within a single country (WAN intra-country), wide area networks across different countries (WAN cross-country), and wide area networks spanning different continents (WAN cross-continent), respectively.

performance discount when using the surrogate model to represent the WF target classifier. Although concept drift occurs, the reduction in performance is not very large, averaging 6.56% on DF set and 8.68% on AWF set. This demonstrates *Stinger's* effectiveness even when the real WF classifier is inaccessible. Moreover, forcing the $\mathcal{P}$ model to defend against various types of surrogate classifiers is a promising approach to mitigate concept drift. However, *Stinger* cannot achieve this goal as it relies on neural surrogate models to compute gradients. This limitation inspires us to explore gradient-independent defenses to improve SDR. Thus exploring the surrogate model gradient-independent method (e.g., reinforcement learning models) is a promising way to further improve *Stinger's* practicality against unknown WFs.

C. Performance Against Adaptive Attacks

Stinger may be vulnerable to adaptive attacks where adversaries recognize its deployment and subsequently modify their classifiers to counteract the data poisoning effects. To address this concern, we evaluate *Stinger's* robustness under such adaptive attack scenarios. Specifically, we construct an adaptive DF classifier by training it on a hybrid dataset comprising both raw and *Stinger*-defended traces. We then evaluate *Stinger's* performance against this adaptive DF attack.

As shown in TABLE VII, the training SDR demonstrates a near-linear correlation with the increasing ratio of *Stinger*-defended traces. This relationship indicates that as more mixed *Stinger*-defended traces are introduced, the WF classifier experiences greater difficulty in model convergence, leading to progressively degraded validation accuracy. Concurrently, the predicted SDR significantly decreases as *Stinger*-defended traces constitute a larger proportion of the training data, suggesting a gradual weakening of the *Stinger's* secret-based defensive efficacy. This is because WF classifiers are more robust to secret changes when the they are exposed to higher volumes of defended traces. The interplay of these two competing factors produces a characteristic non-monotonic trend in the overall SDR, that is, an initial decline is followed by a period of relative stability, culminating in a subsequent increase. Our experimental results demonstrate that adaptive attacks against *Stinger* remain fundamentally limited in effectiveness, achieving only approximately 50%, which is significantly smaller than these of the existing WF classifiers trained on raw traces ($\geq$ 90%) and may fail to provide much meaningful information for adversaries. More detailed results are available at *https://github.com/sxy888/Stinger/blob/main/experiment.md*.

D. Computational Overhead

1) Overhead of $\mathcal{P}$ Model: We evaluate the computational overhead of the $\mathcal{P}$ model in terms of GPU time and memory usage. Our results demonstrate that the $\mathcal{P}$ model is highly efficient during both training and inference. During training, the model converges by the 30th epoch, requiring only 18.12 minutes and 1,495 MB of GPU memory on the *DF* dataset, and 33.48 minutes and 1,493 MB on the *AWF* dataset. Such low resource consumption makes the $\mathcal{P}$ model exceptionally lightweight for offline training. In the inference phase, the $\mathcal{P}$ model exhibits remarkable efficiency, and it utilizes merely 4,632 MB of memory while achieving a latency of 0.75 seconds when generating up to 2^{20} $\mathcal{P}$ sequences. This performance is device-efficient, that is, a single *Nvidia GeForce RTX* 2080 GPU can support secret updates for over 1 million users every 0.75s, underscoring its practicality for large-scale deployments.

2) Overhead of PKI and TLS: We evaluate the overhead of establishing secure sessions between clients and $\mathcal{P}$ model servers, focusing on CPU/memory consumption and runtime latency. Specifically, the maximum memory usage recorded at the $\mathcal{P}$ model server is less than 50 *KB*, while the client reaches peaks of 10 *KB*. These resource requirements remain lightweight on both the server and typical end-user devices. Moreover, the session establishment process incurs minimal overhead, with peak usage remaining below 10% for both the $\mathcal{P}$ model server and clients. The server utilizes an Intel(R) Xeon(R) CPU E5-2630 v4, while the client uses an Intel(R) Core(TM) Ultra 7 258V. The CPU time required for this process is 0.5 ms on the server side and 3.1 ms on the client side, which have negligible impacts on the overall latency.

To assess runtime latency, we deploy the $\mathcal{P}$ model server across multiple geographical locations to evaluate the impact of network distance. The deployment scenarios span local area networks (LAN), wide area networks within a single country (WAN intra-country), wide area networks across different countries (WAN cross-country), and wide area networks spanning different continents (WAN cross-continent). The corresponding latency distributions observed under these network configurations are summarized in TABLE VIII.

As shown in TABLE VIII, the latency incurred during secure session establishment remains relatively low across all tested scenarios. Even under the worst-case cross-continent setting, the maximum observed latency does not exceed 0.5 seconds, a value that can be considered lightweight for such

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
THE PROCESS OF RETRAINING P MODEL ON NEW TRACES

Epoch	1	2	3	4	5	6	7	8	9	10
Time (s)	72	85	83	79	75	75	82	81	80	77
Loss	0.47	0.36	0.32	0.29	0.24	0.19	0.15	0.14	0.14	0.13
Accuracy* (%)	82.75	86.75	88.96	89.7	92.08	93.8	94.51	95.66	95.81	95.94

security-critical operations. The reason is analyzed as follows. The primary purpose of establishing a secure session between the client and the $\mathcal{P}$ server is to create a trusted channel through which the client can securely request secrets for various websites. Importantly, once established, each session remains valid for an extended duration, typically tens of minutes in practice (20 minutes in our implementation). As a result, the one-time latency cost, introduced by the PKI and TLS handshakes, is amortized over the entire session lifetime (0.486/1200), which is negligible value.

3) Overhead of End-End Latency: We measure the latency of processing the $\mathcal{P}$ sequences on the middle node (latency-I) and the increased end-to-end latency due to inserting the $\mathcal{P}$ sequences (latency-II). The results demonstrate that latency-I is negligible, while latency-II is relatively low compared to existing defenses. In particular, with a default $\mathcal{P}$ sequence length of 128, the average latency-I is merely 1.51 ms. This is a very small fraction of the total end-to-end latency, which spans 50-200 ms, making it a minor contribution to the overall delay. Moreover, the average latency-II is 10.6 ms, accounting for approximately 5% to 20% of the total latency, a value that is somewhat high. Nevertheless, *Stinger* remains lightweight compared to existing defenses, as latency-II exhibits a positive correlation with the number of dummy packets under the same deployment architecture, and *Stinger* can achieve a comparable SDR with significantly fewer dummy packets.

E. Robustness Analysis

It is very interesting to investigate the impact of traffic fluctuations on collision rates and the performance of the $\mathcal{P}$ model. To this end, we conduct experiments under the following setup. First, the $\mathcal{P}$ model is trained on traffic traces from 85 websites, achieving a validation accuracy of 96.75%. It is then evaluated on traces from 10 additional websites, where its accuracy drops to 52.7%. This result suggests that website updates can degrade the performance of the $\mathcal{P}$ model, namely, *Stinger*. While the impact of such updates can be mitigated by retraining the $\mathcal{P}$ model, the specific retraining procedure is detailed in TABLE IX.

As shown in TABLE IX, the $\mathcal{P}$ model converges after 10th epoch, requiring 789 seconds of training time, a relatively small cost for the following reasons. First, the experimental setup represents an extreme scenario where approximately 1/8 of the monitored websites totally change simultaneously, a situation unlikely to occur in real-world settings. Second, although long-term continuous updates can indeed degrade the accuracy in the $\mathcal{P}$ model, we find that the average time until a 3σ performance drop occurs is around 10.6 hours, which is substantially longer than 789 seconds.

It is highly valuable to develop strategies that enable the $\mathcal{P}$ model to remain robust against website updates and the emergence of new websites. The key lies in detecting when the $\mathcal{P}$ model becomes ineffective, thereby triggering retraining processes and updating the secret table.

As previously mentioned, validation accuracy of the $\mathcal{P}$ model reflects the optimal performance of *Stinger*, which will be compromised by discrepancies between real WFs and surrogate models in real world scenarios. Therefore, the validation accuracy serves as a suitable metric for assessing the effectiveness of the $\mathcal{P}$ model/sequence against updated or new traffic traces. Specifically, for traces from updated or new websites, we evaluate the $\mathcal{P}$ model's validation accuracy and check whether it falls below the threshold $\mu-3\sigma$, where μ and σ denote the mean and standard deviation of accuracy across previously observed websites (3σ rule in anomaly detection field). If the accuracy is below this threshold, the $\mathcal{P}$ model is retrained to adapt to the changes. Moreover, two distinct strategies are employed depending on the scenario. For new websites, we assess whether model retraining is necessary and act accordingly. For updated websites, a periodic trigger is set to initiate the detection process and determine if retraining is required. Based on one week of monitoring, the average interval for retraining the model is 10.6 hours.

VIII. RELATED WORK

We review related work from WF attacks and WF defenses.

A. WF Attacks

WF attacks aim to identify victims' target websites by intercepting *Tor* traces and analyzing their patterns. Existing WF attacks are broadly categorized into: feature engineering-based traditional attacks and deep learning-based methods.

1) Traditional WF Attacks: Attacks in this category involve feature engineering on the raw traces, followed by training classifiers on these features. These classifiers are typically machine learning models, e.g., ***S**upport **V**ector **M**achines* (***SVM***), ***K**-Nearest **N**eighbors* (***KNN***), and ***R**andom **F**orests* (***RF***). A detailed review of traditional attacks is as follows.

Herrmann et al. [49] exploit packet frequency to classify 775 websites in a closed-world scenario, achieving an accuracy of 3%. Panchenko et al. [37] define fine-grained features based on traffic volume, timing, and direction, reaching an accuracy of 55% on the same trace. Wang et al. [3] use the *KNN* to classify 100 websites based on packet order and the number of incoming/outgoing cells, achieving an accuracy of 91%. *Hayes* proposes *k-FP* [6], which combines *RF* and *KNN* to achieve an accuracy of 95.5%. Panchenko et al. [2] later propose *CUMUL*, which extracts 104 features and feeds them into an *SVM* classifier, resulting in a classification accuracy of 97.3%.

2) Deep WF Attacks: Attacks in this category use deep learning models to fingerprint websites, operating directly on raw traces without any feature engineering. A detailed review is as below.

Sirinam et al. [4] present the first deep learning-based WF attack, known as DF, which employs a ***C**onvolutional **N**eural **N**etwork* (***CNN***) and achieves an accuracy of over 98%. Rimmer et al. propose *AWF* [9], which includes three versions using CNN, ***S**tacked **D**enoising **A**uto-Encoder* (***SDAE***), and ***L**ong **S**hort **T**erm **M**emory* (***LSTM***), each achieving an accuracy close to 90%. *Var-CNN* uses the *ResNet-18* architecture, with early stopping and learning rate decay to enhance attacking performance. Since *Tor* trace patterns can change when websites or the *Tor* protocols update, research has explored WF attacks on few-shot traces. For instance, ***T**riplet **F**ingerprinting* (***TF***) [5] uses a pre-trained network to fingerprint victims' websites, enabling attackers to achieve

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

comparable classification performance with only a few samples. Moreover, *GANDaLF* [10] employs semi-supervised learning with a ***G**enerative **A**dversarial **N**etwork* (***GAN***) [50] to perform WF attacks, which proves effective with limited training traces.

In addition to improving attack performance through more advanced methods, efforts have also been made to enhance the practicality of WFs as follows. Qian et al. [51] propose filter-assisted and augmentation-assisted attacks to improve the robustness of WFs in noisy environments. Bahramali et al. [52] enhance the performance of WF classifiers in unobserved network conditions by augmenting network traces.

B. WF Defenses

WF defenses manipulate or perturb the raw traces by methods such as inserting dummy packets and delaying real packets, thereby masking the distinctive patterns of traffic. The existing defenses commonly share the idea of camouflage, where the raw trace is disguised to resemble either targeted traces or the random ones, effectively disrupting the WF attacks. The existing WF defenses can be broadly categorized into two groups: random camouflage and pair camouflage.

1) Random Camouflage WF Defenses: Defenses in this category protect raw traces through randomization, minimizing the differences between various traces, much like soldiers wearing camouflage uniforms. A detailed review is provided below.

BuFLO [22] strictly controls the rate and interval of packets, effectively masking the timing features of raw traces. However, it suffers from high bandwidth overhead and significant latency in practice. Improved versions of *BuFLO* have since been proposed, including *Tamaraw* [20] and *CS-BuFLO* [12], which are designed to mitigate these bandwidth and latency issues. Additionally, *WTF-PAD* [21] uses adaptive padding to randomize raw traces by injecting dummy packets, thereby approximating the timing distribution of normal traces. *FRONT* [17] takes a different approach by injecting more dummy packets at the beginning of the trace and randomizing the number of these packets. Since random camouflage defenses are not tailed for specific classifier, they suffer from high bandwidth overheads.

2) Pair Camouflage WF Defenses: Defenses in this category camouflage the raw traces of a particular website to resemble another targeted one, thereby confusing WF classifiers. The detailed review of pair camouflage is as below.

Walkie-Talkie [11] forces *Tor* clients and servers to use half-duplex communication, creating a super-sequence of two trace bursts to confuse classifiers. However, this approach is difficult to implement, and Gong and Wang [17] have shown that *Walkie-Talkie* is ineffective against deep learning-based attacks. *Mockingbird* [19] uses distance gradient descent to make one trace to resemble another. ***A**dversarial **W**ebsite **A**daptation* (***AWA***) [18] creates a transformer set using feedback from an auxiliary classifier and discriminator. Each transformer generates adversarial traces to evade the WF classifier. Compared to random camouflage, pair camouflage is more bandwidth-efficient because it is tailored for specific traces. However, these methods still suffer from the defense dilemma: improving defense performance often comes at the cost of increased bandwidth overhead, making them less practical.

In addition to reducing bandwidth overhead, efforts have also been made to resist adversarial examples, which pose significant threats to the reliability of WF defenses. For instance, Qiao et al. [53] propose *Acup* as a defense mechanism against WF attacks enhanced with adversarial training. *Acup* sequentially increases the difficulty of website classification, generates trace-agnostic perturbations, and employs perturbation variation to diversify the traffic traces of different users.

IX. Conclusion

WF defenses protect *Tor* users against website fingerprinting attacks by perturbing raw traces. Existing camouflage-based defenses often face such a dilemma: their defense performance increases at the cost of higher bandwidth overhead, making them less practical. In this work, we propose *Stinger*, which actively compromises WF classifiers by inserting special triggers into adversaries' training traces. Experimental results demonstrate *Stinger's* efficiency, showing comparable defense performance while significantly reducing bandwidth overhead. The prototype system integrated *Stinger* also demonstrates its high practicality in real-world settings. *Stinger* offers a novel perspective by actively poisoning adversaries' models to achieve better performance with lower defense costs.

References

[1] *Users-Tor Metrics*. Accessed: Nov. 2, 2022. [Online]. Available: https://metrics.torproject.org/userstats-relay-country.html

[2] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Jun. 2016.

[3] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. 23rd USENIX Secur. Symp. (USENIX Secur.)*, 2014, pp. 143–157.

[4] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943.

[5] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1131–1148.

[6] J. Hayes and G. Danezis, "K-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. USENIX Secur. Symp.*, May 2015, pp. 1187–1203.

[7] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 4, pp. 292–310, Oct. 2019.

[8] T. Wang, "High precision open-world website fingerprinting," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 152–167.

[9] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Sep. 2018.

[10] S. E. Oh, N. Mathews, M. S. Rahman, M. Wright, and N. Hopper, "GANDaLF: GAN for data-limited fingerprinting," *Proc. Privacy Enhancing Technol.*, vol. 2021, no. 2, pp. 305–322, Apr. 2021.

[11] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. 26th USENIX Secur. Symp. (SEC)*, Vancouver, BC, Canada, Aug. 2017, pp. 1375–1390.

[12] X. Cai, R. Nithyanand, and R. Johnson, "CS-BuFLO: A congestion sensitive website fingerprinting defense," in *Proc. 13th Workshop Privacy Electron. Soc.*, Nov. 2014, pp. 121–130.

[13] M. Nasr, A. Bahramali, and A. Houmansadr, "Blind adversarial network perturbations," 2020, *arXiv:2002.06495*.

[14] A. Abusnaina, R. Jang, A. Khormali, D. Nyang, and D. Mohaisen, "DFD: Adversarial learning-based approach to defend against website fingerprinting," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, Toronto, ON, Canada, Jul. 2020, pp. 2459–2468.

[15] C. Hou, G. Gou, J. Shi, P. Fu, and G. Xiong, "WF-GAN: Fighting back against website fingerprinting attack using adversarial learning," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2020, pp. 1–7.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.

[16] W. De la Cadena et al., "TrafficSliver: Fighting website fingerprinting attacks with traffic splitting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, New York, NY, USA, Nov. 2020, pp. 1971–1985.
[17] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. 29th USENIX Secur. Symp. (USENIX Security)*, 2020, pp. 717–734.
[18] A. M. Sadeghzadeh, B. Tajali, and R. Jalili, "AWA: Adversarial website adaptation," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3109–3122, 2021.
[19] M. S. Rahman, M. Imani, N. Mathews, and M. Wright, "Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1594–1609, 2021.
[20] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 227–238.
[21] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. 21st Eur. Symp. Res. Comput. Secur. (ESORICS)*, Heraklion, Greece, Sep. 2016, pp. 27–46.
[22] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. Secur. Privacy*, May 2012, pp. 332–346.
[23] G. Huang et al., "Efficient and low overhead website fingerprinting attacks and defenses based on TCP/IP traffic," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 1991–1999.
[24] L. Qiao, B. Wu, H. Li, C. Gao, W. Yuan, and X. Luo, "Trace-agnostic and adversarial training-resilient website fingerprinting defense," in *Proc. IEEE Conf. Comput. Commun. (IEEE INFOCOM)*, May 2024, pp. 211–220.
[25] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, "A critical evaluation of website fingerprinting attacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 263–274.
[26] J. Alcock, *Animal Behavior: An Evolutionary Approach*. MA, USA: Sinauer Associates, 2009.
[27] Y. He, G. Meng, K. Chen, X. Hu, and J. He, "Towards security threats of deep learning systems: A survey," *IEEE Trans. Softw. Eng.*, vol. 48, no. 5, pp. 1743–1770, May 2022.
[28] J. Lin, L. Xu, Y. Liu, and X. Zhang, "Composite backdoor attack for deep neural network by mixing existing benign features," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Apr. 2020, pp. 113–131.
[29] X. Cao, J. Jia, and N. Z. Gong, "Data poisoning attacks to local differential privacy protocols," in *Proc. USENIX Secur. Symp.*, Jun. 2019, pp. 947–964.
[30] E. Bagdasaryan and V. Shmatikov, "Adversarial illusions in multi-modal embeddings," in *Proc. USENIX Secur. Symp.*, 2023, pp. 3009–3025.
[31] W. Ding, A. N. Bhagoji, B. Y. Zhao, and H. Zheng, "Towards scalable and robust model versioning," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Apr. 2024, pp. 592–611.
[32] A. Demonti et al., "Why do adversarial attacks transfer? Explaining transferability of evasion and poisoning attacks," in *Proc. USENIX Security Symp.*, 2019, pp. 321–338.
[33] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," in *Proc. Int. Conf. Learn. Represent.*, 2017.
[34] J. Gong, W. Zhang, C. Zhang, and T. Wang, "WFDefProxy: Real world implementation and evaluation of website fingerprinting defenses," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1357–1371, 2024.
[35] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, "Tik-tok: The utility of packet timing in website fingerprinting attacks," *Proc. Privacy Enhancing Technol.*, vol. 2020, no. 3, pp. 5–24, Jul. 2020.
[36] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.
[37] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. 10th Annu. ACM Workshop Privacy Electron. Soc.*, Oct. 2011, pp. 103–114.
[38] M. Beckerle, J. Magnusson, and T. Pulls, "Splitting hairs and network traces: Improved attacks against traffic splitting as a website fingerprinting defense," in *Proc. 21st Workshop Privacy Electron. Soc.*, Nov. 2022, pp. 15–27.
[39] N. Mathews, J. K. Holland, N. Hopper, and M. Wright, "Laserbeak: Evolving website fingerprinting attacks with attention and multi-channel feature representation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9285–9300, 2024.
[40] S. Han, C. Lin, C. Shen, Q. Wang, and X. Guan, "Interpreting adversarial examples in deep learning: A review," *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–38, Dec. 2023.
[41] A. Serban, E. Poll, and J. Visser, "Adversarial examples on object recognition: A comprehensive survey," *ACM Comput. Surv.*, vol. 53, no. 3, pp. 1–38, May 2021.
[42] R. R. Wiyatno, A. Xu, O. Dia, and A. de Berker, "Adversarial examples in modern machine learning: A review," 2019, *arXiv:1911.05268*.
[43] J. Zhang and C. Li, "Adversarial examples: Opportunities and challenges," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 7, pp. 2578–2593, Jul. 2020.
[44] A. S. Ross and F. Doshi-Velez, "Improving the adversarial robustness and interpretability of deep neural networks by regularizing their input gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 32, no. 1, pp. 1660–1669.
[45] K. Nar, O. Ocal, S. S. Sastry, and K. Ramchandran, "Cross-entropy loss and low-rank features have responsibility for adversarial examples," 2019, *arXiv:1901.08360*.
[46] C.-J. Simon-Gabriel, Y. Ollivier, L. Bottou, B. Schölkopf, and D. Lopez-Paz, "First-order adversarial vulnerability of neural networks and input dimension," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Aug. 2018, pp. 5809–5817.
[47] K. Grosse, P. Manoharan, N. Papernot, M. Backes, and P. McDaniel, "On the (statistical) detection of adversarial examples," 2017, *arXiv:1702.06280*.
[48] D. I. Diochnos, S. Mahloujifar, and M. Mahmoody, "Adversarial risk and robustness: General definitions and implications for the uniform distribution," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 10359–10368.
[49] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: Attacking popular privacy enhancing technologies with the multinomial Naïve–Bayes classifier," in *Proc. ACM Workshop Cloud Comput. Secur.*, Nov. 2009, pp. 31–42.
[50] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Proc. Neural Inf. Process. Syst. (NIPS)*, 2016, pp. 901–909.
[51] Y. Qian et al., "Enhancing resilience in website fingerprinting: Novel adversary strategies for noisy traffic environments," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7216–7231, 2024.
[52] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1035–1049.
[53] L. Qiao, B. Wu, S. Yin, H. Li, W. Yuan, and X. Luo, "Resisting DNN-based website fingerprinting attacks enhanced by adversarial training," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5375–5386, 2023.



Lihai Nie received the bachelor's and master's degrees from Dalian Maritime University, China, in 2015 and 2018, respectively. He is currently pursuing the Ph.D. degree with the College of Intelligence and Computing, Tianjin University, China. He is a Post-Doctoral Researcher with the College of Cyber Science, Nankai University, China. His research interests include both network security and machine learning.

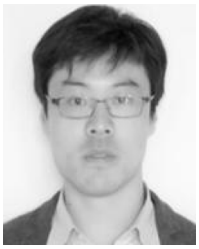


Xiaodong Dong (Member, IEEE) received the bachelor's and master's degrees from Northwestern Polytechnical University, Xi'an, China, in 2013 and 2016, respectively, and the Ph.D. degree from the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University, Tianjin, China, in 2020. Currently, he is a Lecturer with the College of Computer Science, Nankai University, Tianjin. His research interests include computer networks, the future internet, and software-defined networking.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.



Lili Shi received the bachelor's degree from Jilin Agricultural University, China, in 2020, and the master's degree from Tianjin University, China, in 2023. Her research interests include cybersecurity and machine learning.



Laiping Zhao received the B.S. and M.S. degrees from Dalian University of Technology, China, in 2007 and 2009, respectively, and the Ph.D. degree from the Department of Informatics, Kyushu University, Japan, in 2012. He is currently a Professor with the College of Intelligence and Computing, Tianjin University. His research interests include cloud computing and network security.



Zheli Liu received the B.S. and M.S. degrees in computer science and the Ph.D. degree in computer application from Jilin University, China, in 2002, 2005, and 2009, respectively. After a Post-Doctoral Fellowship with Nankai University, he joined the College of Cyber Science, Nankai University, in 2011. Currently, he is with Nankai University as a Professor. His current research interests include applied cryptography and data privacy protection.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:18:15 UTC from IEEE Xplore. Restrictions apply.